Machine Learning

Contents

1 Linear Regression

1.1 Simple Linear Regression

- Simple linear regression fits a straight line to data to explain how one variable (the predictor) influences another (the response)

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- The above equation is the population regression equation;
- The unknown coefficients β_0 and β_1 in the population regression line is unknown. We estimate these unknown coefficients using the principle of least squares.
  To estimate the unknown coefficients, we minimize the **Residual Sum of Squares (RSS)**:

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Since

$$\hat{y}_i = \beta_0 + \beta_1 x_i,$$

the objective function becomes

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

We obtain the least squares estimates by differentiating RSS with respect to β_0 and β_1, and setting the derivatives equal to zero.

Step 1: Differentiate with respect to β_0

$$\frac{\partial RSS}{\partial \beta_0} = -2 \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\Rightarrow \sum_{i=1}^{n} y_i = n\beta_0 + \beta_1 \sum_{i=1}^{n} x_i$$

Dividing by n,

$$\bar{y} = \beta_0 + \beta_1 \bar{x}$$

Hence,

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Step 2: Differentiate with respect to β_1

$$\frac{\partial RSS}{\partial \beta_1} = -2 \sum_{i=1}^{n} x_i (y_i - \beta_0 - \beta_1 x_i) = 0$$

Substituting $\beta_0 = \bar{y} - \beta_1 \bar{x}$ and simplifying,

$$\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) = \beta_1 \sum_{i=1}^{n} (x_i - \bar{x})^2$$

Therefore,

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}$$

Thus, the least squares estimates are

$$\boxed{\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}}$$

$$\boxed{\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}}$$

Hence, the fitted regression line is

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X$$

1.2 Correlation and Covariance

- **Covariance** measures whether two variables move together. A positive covariance means they increase or decrease together, while a negative covariance means they move in opposite directions.

$$cov(X, Y) = \frac{1}{N} \sum_{i=1}^{N} (X_i - \bar{X})(Y_i - \bar{Y})$$

- The numerator term appears frequently in statistics and regression, so it is commonly written as:

$$S_{xy} = \sum_{i=1}^{N} (X_i - \bar{X})(Y_i - \bar{Y})$$

Hence:

$$cov(X, Y) = \frac{S_{xy}}{N}$$

- Similarly, the variance-related quantities are written as:

$$S_{xx} = \sum_{i=1}^{N} (X_i - \bar{X})^2$$

$$S_{yy} = \sum_{i=1}^{N} (Y_i - \bar{Y})^2$$

where:

S_{xx} measures variation in X

S_{yy} measures variation in Y

S_{xy} measures joint variation between X and Y

- The **magnitude of covariance** depends on the scale of the variables, making it difficult to compare relationships across datasets.
- **Correlation** is the normalized form of covariance. It removes scale dependence by dividing covariance by the product of standard deviations.

$$r = \frac{cov(X, Y)}{\sqrt{var(X) var(Y)}}$$

Using the S-notation:

$$r = \frac{S_{xy}}{\sqrt{S_{xx} S_{yy}}}$$

- Since correlation is normalized, it always lies in the range:

$$-1 \leq r \leq 1$$

- Interpretation of correlation:
  - $r = +1$: perfect positive linear relationship
  - $r = -1$: perfect negative linear relationship
  - $r = 0$: no linear relationship

- Covariance and correlation describe association, not causation. A strong correlation does not imply that one variable causes the other.
- The S_{xx}, S_{yy}, S_{xy} notation is extremely important in linear regression because the slope estimator can be written compactly as:

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}$$

This provides an intuitive interpretation:

$$\text{Slope} = \frac{\text{joint variation}}{\text{variation in } X}$$

1.3 Multiple Linear Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon$$

Here, each coefficient β_j measures the effect of predictor X_j on Y, holding all other variables constant.

$$Y = X\beta + \varepsilon,$$

where X is the design matrix containing all predictors (with a column of ones for the intercept).

- The goal is to find the parameter vector $\hat{\beta}$ that minimizes the sum of squared residuals:

$$\min_{\beta} \|Y - X\beta\|^2.$$

- When $X'X$ is invertible, the closed-form solution for the regression coefficients is:

$$\hat{\beta} = (X'X)^{-1}X'Y.$$

- The matrix $X'X$ is known as the **Gram matrix**. It encodes all pairwise inner products between features and:
  - Determines how correlated or redundant the predictors are.
  - Must be invertible for a unique solution to exist.
  - Can become ill-conditioned when predictors are highly collinear.
- This closed-form estimator is called the **Ordinary Least Squares (OLS)** solution and provides the best linear unbiased estimator (BLUE) under the Gauss–Markov assumptions.

Consider the multiple linear regression model: $\hat{y} = Xw$, where $X \in \mathbb{R}^{n\times d}$ is the feature matrix, $w \in \mathbb{R}^{d\times 1}$ is the parameter vector, and $y \in \mathbb{R}^{n\times 1}$ is the target vector. The Residual Sum of Squares (RSS) loss is: $RSS = (y - Xw)^\top (y - Xw)$

Step 1: Expand the Quadratic Form We use the matrix identity:

$$(a-b)^\top (a-b) = a^\top a - a^\top b - b^\top a + b^\top b$$

Then:

$$RSS = y^\top y - y^\top Xw - (Xw)^\top y + (Xw)^\top (Xw)$$

Now simplify each term.
Using the transpose identity:

$$(AB)^\top = B^\top A^\top$$

Thus:

$$RSS = y^\top y - y^\top Xw - w^\top X^\top y + w^\top X^\top Xw$$

Step 2: Use Scalar Transpose Property A very important identity is: $s^\top = s$ for any scalar s.

$$RSS = y^\top y - 2w^\top X^\top y + w^\top X^\top Xw$$

Step 3: Differentiate with Respect to w **Identity 1:**

$$\frac{\partial}{\partial w}(a^\top w) = a$$

where a is a constant vector.

Identity 2:

$$\frac{\partial}{\partial w}(w^\top A w) = (A + A^\top)w$$

where A is a constant matrix.
If A is symmetric: $A^\top = A$
First term:

$$\frac{\partial}{\partial w}(y^\top y) = 0$$

because it does not depend on w.

Second term:

$$\frac{\partial}{\partial w}(-2w^\top X^\top y) = -2X^\top y$$

Third term:

$$\frac{\partial}{\partial w}(w^\top X^\top X w)$$

Let:

$$A = X^\top X$$

Notice:

$$(X^\top X)^\top = X^\top X$$

Hence $X^\top X$ is symmetric.
Therefore:

$$\frac{\partial}{\partial w}(w^\top X^\top X w) = 2X^\top X w$$

Combining all terms:

$$\frac{\partial RSS}{\partial w} = -2X^\top y + 2X^\top X w$$

Step 4: Set Gradient Equal to Zero

$$-2X^\top y + 2X^\top X w = 0$$

Divide both sides by 2:

$$-X^\top y + X^\top X w = 0$$

Rearranging:

$$X^\top X w = X^\top y$$

These are called the **Normal Equations**.

Step 5: Solve for w If $X^\top X$ is invertible:

$$\boxed{\hat{w} = (X^\top X)^{-1} X^\top y}$$

1.4 $X'X$, $(X'X)^{-1}$, and $X'Y$

- $X'X$
  - $X'X$ computes all pairwise inner products between features. Each entry $(X'X)_{ij}$ equals $\langle X_i, X_j \rangle$, the dot product between feature i and feature j.
  - This matrix captures how similar, correlated, or redundant the predictors are.
  - Large off-diagonal values mean two features move together strongly (collinearity).
  - So $X'X$ is essentially a "feature correlation structure" or a geometric summary of how the predictors relate in feature space.
- $(X'X)^{-1}$
  - The inverse untangles the correlations between predictors.
  - It removes redundancy and adjusts each coefficient so that the contribution of a predictor is measured *while holding all other predictors fixed*.
  - If two features are highly correlated, $(X'X)^{-1}$ becomes unstable, which is why multicollinearity is dangerous: the model struggles to isolate individual effects.
  - Geometrically, $(X'X)^{-1}$ transforms the correlated predictor space into an orthogonal (uncorrelated) coordinate system.
- $X'Y$
  - $X'Y$ computes the inner product between each feature and the response vector.
  - The entry $(X'Y)_j$ equals $\langle X_j, Y \rangle$, measuring how strongly predictor j aligns with or explains the variation in Y.
  - This vector captures the relationship between each predictor and the target before adjusting for interactions with other predictors.
  - In simple regression, $X'Y$ is directly related to covariance between X and Y.
- **Putting it all together:**
  - $X'Y$ tells you "how each feature relates to Y".
  - $X'X$ tells you "how the features relate to each other".
  - $(X'X)^{-1}$ adjusts for feature correlations so we correctly isolate each feature's unique contribution to predicting Y.

1.5 Linear Regression Using Gradient Descent

1.5.1 Derivation of Gradient – Matrix Form

Consider the linear regression model:

$$\hat{y}_i = w^\top x_i + b$$

where:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}, \qquad x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix}$$

The Residual Sum of Squares (RSS) loss is defined as:

$$RSS = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Substituting the prediction equation:

$$RSS = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (w^\top x_i + b) \right)^2$$

We now derive the gradients with respect to the weights and bias.

Gradient with Respect to Weight w_k We compute:

$$\frac{\partial RSS}{\partial w_k} = \frac{\partial}{\partial w_k}\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2\right]$$

Using linearity of differentiation:

$$= \frac{1}{n}\sum_{i=1}^{n}\frac{\partial}{\partial w_k}(y_i - \hat{y}_i)^2$$

Using the chain rule:

$$\frac{d}{dx}(u^2) = 2u\frac{du}{dx}$$

Let:

$$u = y_i - \hat{y}_i$$

Then:

$$= \frac{1}{n}\sum_{i=1}^{n}2(y_i - \hat{y}_i)\frac{\partial}{\partial w_k}(y_i - \hat{y}_i)$$

Since y_i is constant with respect to w_k:

$$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial w_k}$$

Now:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_k x_{ik} + \cdots + b$$

Therefore:

$$\frac{\partial \hat{y}_i}{\partial w_k} = x_{ik}$$

Substituting:

$$\boxed{\frac{\partial RSS}{\partial w_k} = -\frac{2}{n}\sum_{i=1}^{n}x_{ik}(y_i - \hat{y}_i)}$$

Repeating this for all weights and stacking the results into a vector gives:

$$\boxed{\frac{\partial RSS}{\partial w} = -\frac{2}{n}X^\top(y - \hat{y})}$$

where:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

and:

$$y - \hat{y} = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

Gradient with Respect to Bias b Now compute:

$$\frac{\partial RSS}{\partial b} = \frac{\partial}{\partial b}\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2\right]$$

Using linearity:

$$= \frac{1}{n}\sum_{i=1}^{n}\frac{\partial}{\partial b}(y_i - \hat{y}_i)^2$$

Applying the chain rule:

$$= \frac{1}{n}\sum_{i=1}^{n}2(y_i - \hat{y}_i)\frac{\partial}{\partial b}(y_i - \hat{y}_i)$$

Since y_i is constant:

$$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial b}$$

From:

$$\hat{y}_i = w^\top x_i + b$$

we obtain:

$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

Therefore:

$$\boxed{\frac{\partial RSS}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)}$$

Important Intuition The feature values appear in the weight gradient because:

$$\frac{\partial \hat{y}_i}{\partial w_k} = x_{ik}$$

However, the bias gradient does not contain feature values because:

$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

Thus, the bias behaves like a parameter multiplied by a constant feature whose value is always 1.

Gradient Descent provides an iterative optimization approach to estimate the regression coefficients instead of using the closed-form solution $(X^\top X)^{-1}X^\top y$. This is particularly useful when the number of features is very large or when matrix inversion becomes computationally expensive.

1.5.2 Update Rule & Formula

- **Goal**: Minimize the Residual Sum of Squares (RSS)

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

  where $\hat{y}_i = w^\top x_i + b$.
- **Idea**: Start with initial guesses for w and b, then repeatedly update them in the direction that decreases the loss.
- **Gradient Computation**:

$$\frac{\partial RSS}{\partial w} = -\frac{2}{n} X^\top (y - \hat{y}), \qquad \frac{\partial RSS}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)$$

- **Parameter Updates**:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial RSS}{\partial w}, \qquad b_{\text{new}} = b_{\text{old}} - \alpha \frac{\partial RSS}{\partial b}$$

  where α is the learning rate.
- **Learning Rate**: Controls the step size. Too large $\rightarrow$ divergence. Too small $\rightarrow$ slow convergence.
- **Stopping Criteria**: Gradient descent continues until:
  - parameters converge, or
  - improvement in loss becomes negligible, or
  - maximum iterations reached.
- **Advantages**:
  - Scales well to very large datasets (big n and big p).
  - Avoids matrix inversion.
  - Works well for online/streaming data.
- **Limitations**:
  - Requires tuning of the learning rate.
  - May converge slowly or get stuck in plateaus.
  - Performance depends on feature scaling—standardization is often required.

In summary, gradient descent offers a flexible and scalable way to fit linear regression, especially when closed-form solutions are computationally challenging.

1.6 Regularization: Ridge, Lasso, and Elastic Net

Regularization methods modify the ordinary least squares (OLS) objective by adding a penalty on the size of the coefficients. This helps control model complexity, reduce overfitting, and handle multicollinearity.

Informally, a norm is a function that accepts as input a vector from our vector space V and spits out a real number that tells us how big that vector is. In order for a function to qualify as a norm, it must first fulfill some properties, so that the results of this metrization process kind of "make sense". These properties are the following.

- Positive Definite
- Absolute scalable
- Triangular Inequality

The Euclidean norm (or L^2-norm) of a vector

$$x = (x_1, x_2, \ldots, x_n)$$

is defined as

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

More generally, the p-norm of a vector is

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

The Euclidean norm is obtained when $p = 2$.

- **Ridge Regression (L2 Regularization)**

$$\hat{\beta}_{ridge} = \arg\min_{\beta} \left(\|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right)$$

  - Penalizes the sum of squared coefficients.
  - Shrinks coefficients smoothly toward zero but never sets them exactly to zero.
  - Very useful when predictors are highly correlated.
  - Objective : $\min_\beta \|y - X\beta\|^2 + \lambda\|\beta\|^2$
  - Closed Form Solution : $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$
  - Gradient Descent Solution : $\nabla = -2X^T(y - X\beta) + 2\lambda\beta$
- **Lasso Regression (L1 Regularization)**

$$\hat{\beta}_{lasso} = \arg\min_{\beta} \left(\|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right)$$

  - Penalizes the sum of absolute values of coefficients.
  - Produces sparse models by forcing some coefficients exactly to zero.
  - Useful for feature selection.
  - Objective : $\min_\beta \|y - X\beta\|^2 + \lambda\|\beta\|_1$
  - Closed Form Solution : No closed-form solution
  - Gradient (Subgradient) : $\nabla = -2X^T(y - X\beta) + \lambda \,\text{sign}(\beta)$
- **Elastic Net (Combination of L1 and L2)**

$$\hat{\beta}_{EN} = \arg\min_{\beta} \left(\|Y - X\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2 \right)$$

  - Combines Ridge and Lasso penalties.
  - Useful when features are correlated and we also want sparsity.
  - More stable than Lasso alone, especially in high-dimensional settings.
  - Objective : $\min_\beta \|y - X\beta\|^2 + \lambda_1\|\beta\|_1 + \lambda_2\|\beta\|^2$
  - Closed Form Solution : No closed-form solution
  - Gradient Descent Solution :$\nabla = -2X^T(y - X\beta) + \lambda_1 \text{sign}(\beta) + 2\lambda_2\beta$

Regularization helps prevent overfitting, improves generalization, and makes models more robust by controlling how large the coefficients are allowed to grow.

- **Why reducing coefficient sizes reduces model complexity**
  - Large coefficients make the model highly sensitive to small changes in the input features, causing unstable predictions and overfitting.
  - By shrinking coefficients toward zero, regularization forces the model to rely only on strong, stable patterns in the data rather than noise.
  - Smaller weights correspond to smoother decision boundaries and simpler hypothesis functions, improving generalization.
  - Thus, controlling coefficient magnitude directly controls the effective complexity of the model.

1.7 Standard Errors of Estimates

$$SE(\hat{\beta}_j) = RSE \cdot \sqrt{(X^\top X)^{-1}_{jj}}$$

- **What the standard error represents**
  - The standard error $SE(\hat{\beta}_j)$ measures the uncertainty in the estimated coefficient $\hat{\beta}_j$.
  - It tells us how much $\hat{\beta}_j$ would vary if we repeatedly sampled new datasets from the same population.
  - A large standard error means the coefficient estimate is unstable or poorly supported by data.
- **Role of the residual standard error (RSE)**
  - The term RSE^2 represents how much noise remains unexplained by the model.
  - Higher noise inflates the standard errors because it becomes harder to attribute variation in Y to any specific predictor.
- **Role of** $(X^\top X)^{-1}$

  - The diagonal element $\left((X^\top X)^{-1}\right)_{jj}$ measures how much information we have about predictor j.
  - If predictors are highly correlated or do not vary much, this term becomes large, reflecting increased uncertainty.
  - Thus, multicollinearity directly causes inflated standard errors.
- **Why we care about standard errors**
  - They allow us to compute confidence intervals for coefficients.
  - They are essential for hypothesis testing (e.g., is $\beta_j = 0$?).
  - Small standard errors mean we trust the coefficient estimates; large ones indicate that the effect may be unreliable.

1.8 Estimate of the sd of the error term : RSE

The Residual Standard Error (RSE) is an estimate of the standard deviation of the error term ,σ in the regression model:

$$RSE = \sqrt{\frac{1}{n-p-1}RSS} = \sqrt{\frac{1}{n-p-1}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

1.9 Coefficient Interpretation - Example

Consider the following multiple linear regression model for annual salary:

$$\text{Salary} = \beta_0 + \beta_1(\text{Experience}) + \beta_2(\text{textHours}) + \beta_3(\text{Gender}) + \beta_4(\text{Masters}) + \beta_5(\text{PhD})$$

- Experience = years of work experience (numerical)
- Hours = hours worked per week (numerical)
- Gender = binary variable (1 = Male, 0 = Female)
- Masters = 1 if the individual holds a Master's degree, 0 otherwise
- PhD = 1 if the individual holds a PhD, 0 otherwise

Bachelor's degree is treated as the baseline education category.

Suppose the estimated regression equation is:

$$\text{Salary} = 200k + 50k(\text{Experience}) + 2k(\text{Hours}) + 30k(\text{Gender}) + 80k(\text{Masters}) + 150k(\text{PhD})$$

- Intercept ($\beta_0 = 200000$):The intercept represents the predicted salary when all explanatory variables are equal to zero. In this case, it corresponds to a female individual with a Bachelor's degree, zero years of experience, and zero working hours. While not always practically meaningful, it serves as a baseline for the regression model.
- Experience ($\beta_1 = 50000$): Holding all other variables constant, a one-year increase in experience is associated with an increase of (50,000) in annual salary.
- Hours Worked ($\beta_2 = 2000$): Holding all other variables constant, each additional hour worked per week is associated with an increase of (2,000) in annual salary.
- Gender ($\beta_3 = 30000$):Holding all other variables constant, male individuals earn, on average, (30,000) more than female individuals.
- Education: Master's ($\beta_4 = 80000$):Holding all other variables constant, individuals with a Master's degree earn (80,000) more than those with a Bachelor's degree (the baseline category).
- Education: PhD ($\beta_5 = 150000$):Holding all other variables constant, individuals with a PhD earn (150,000) more than those with a Bachelor's degree.

1.9.1 Effect of Changing Units

The numerical values of regression coefficients depend on the units in which variables are measured. However, the underlying relationship and model predictions remain unchanged.

- **Changing Units of an Input Variable**: If $Experience$ is measured in months instead of years (1 year = 12 months), the coefficient adjusts accordingly:

$$\beta_1^{(\text{months})} = \frac{50000}{12} \approx 4167$$

Thus, the interpretation becomes: a one-month increase in experience is associated with an increase of approximately (4,167) in salary.

- **Rescaling the Response Variable** If salary is expressed in lakhs of rupees (1 lakh = 100,000), the regression equation becomes:

$$\text{Salary} = 2 + 0.5(\text{Experience}) + 0.02(\text{Hours}) + 0.3(\text{Gender}) + 0.8(\text{Masters}) + 1.5(\text{PhD})$$

All coefficients are scaled proportionally, but their interpretations remain consistent within the new unit system.

1.10 R^2 Statistics

The R^2 statistic measures the proportion of variability in the response variable y that is explained by the regression model using the predictors X. It quantifies how much better the regression model is compared to simply predicting the mean of y.

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}$$

- **Total Sum of Squares (TSS)**

$$TSS = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

  - Measures the total variability of y around its mean.
  - Represents how spread out the data is before fitting any model.
- **Residual Sum of Squares (RSS)**

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

  - Measures remaining variability after fitting the model.
  - Small RSS means the model fits the data well.
- **Explained Sum of Squares (ESS)** (Optional but useful)

$$ESS = \sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2$$

  - Measures how much of the total variability the model is able to explain.
- **Interpretation of R^2**
  - $R^2 = 0$ means the model explains none of the variability—no better than predicting the mean.
  - $R^2 = 1$ means perfect prediction.
  - Higher R^2 means the fitted regression line captures more of the variation in y.

1.11 Metrics from L-P Norm

$$MAE = \frac{1}{m} \sum_{i=1}^{m} |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

1.12 Huber Loss

$$\begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta \cdot (|y - f(x)| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

The Huber loss is more robust to outliers than MSE because it reduces the impact of large errors on the loss. It avoids the large errors squared in the quadratic term that can dominate the loss in the presence of outliers, making it more resistant to the influence of data points with extremely high residuals. The choice of the δ parameter depends on the specific characteristics of the data and the desired balance between the properties of MSE and MAE. A smaller δ makes the loss more quadratic, while a larger δ makes it more linear.

In summary, Huber loss is a compromise loss function that combines the advantages of both mean squared error and mean absolute error, providing a good balance between sensitivity to outliers and differentiability.

1.13 Adjusted R^2

Adjusted R^2 is a modification of the usual R^2 that accounts for the number of predictors used in the model. While R^2 always increases (or at least never decreases) when more variables are added, Adjusted R^2 increases only if a new predictor improves the model more than would be expected by chance.

$$\text{Adjusted}R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

- **Why we need Adjusted R^2**
  - Adding a new predictor always reduces RSS, which makes R^2 artificially increase—even if the predictor is useless.
  - Adjusted R^2 corrects this by introducing a penalty for adding more features.
  - It allows fair comparison between models with different numbers of predictors.
- **How the penalty works**
  - The denominator $N - p - 1$ shrinks as the number of predictors p grows.
  - This makes the fraction larger, which decreases Adjusted R^2 when unnecessary predictors are added.
  - Only predictors that substantially reduce RSS will cause Adjusted R^2 to increase.
- **Interpretation**
  - Higher Adjusted R^2 indicates a model that explains variability efficiently, without relying on too many predictors.
  - If Adjusted R^2 decreases when a new variable is added, that variable is likely not contributing useful information.

1.14 Feature Significance - T test

In linear regression, we often want to test whether a particular predictor X_1 has a meaningful effect on the response variable Y. This is done through a hypothesis test on its coefficient β_1.

$$H_0 : \beta_1 = 0 \quad \text{(no relationship between } X_1 \text{ and } Y\text{)}$$
$$H_1 : \beta_1 \neq 0 \quad \text{(some relationship exists)}$$

To test this, we compute the t–statistic:

$$t = \frac{\hat{\beta}_1 - 0}{SE(\hat{\beta}_1)}$$

- The numerator measures how far the estimated coefficient is from the null hypothesis value.
- The denominator (standard error) reflects the uncertainty in the estimate. Large uncertainty makes it harder to claim significance.
- A large $|t|$ indicates that $\hat{\beta}_1$ is far from zero relative to its noise level.
- We compare this t value with a t-distribution (with $n - p - 1$ degrees of freedom) to compute a p-value.
- A small p-value means the observed effect is unlikely under H_0, leading us to reject the null hypothesis.

This test helps determine whether a predictor contributes meaningful explanatory power to the model.

1.15 Overall Significance - F Test

- Is at least one of the predictors $X_1, X_2, ..., X_p$ useful in predicting the response?
- Do all the predictors help to explain Y, or is only a subset of the predictors useful?
- $H_o : \beta_1 = \beta_2 = ... = \beta_p = 0$ versus the alternative
- H_a : at least one β_j is non-zero.

$$F = \frac{(TSS - RSS)/p}{RSS/(n - p - 1}$$

- IF f statistic takes a value close to 1, then we cannot expect any relationship between the response and predictors.
- Larger the f value higher the the evidence against the null hypothesis.
- Sometimes, we want to test that a particular subset of q of the coefficients are zero.

- In this case we fit two model and compute their respective RSS value(with and without that particular subset).

$$F = \frac{(RSS_{\text{reduced}} - RSS_{\text{full}})/q}{RSS_{\text{full}}/(n-p-1)}$$

1.16 Confidence Intervals of the estimates

Confidence intervals and prediction intervals are essential tools in regression analysis for quantifying the uncertainty in estimates and predictions. While point estimates (e.g., $\hat{\beta}_j$ or $\hat{y}$) give single best guesses, these intervals express how confident we are about those estimates based on the available data.

For each coefficient β_j in the linear model

$$Y = X\beta + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I),$$

we can construct a confidence interval (CI) for the estimated coefficient $\hat{\beta}_j$.

$$\hat{\beta}_j \pm t_{\alpha/2,\, n-p-1} \cdot SE(\hat{\beta}_j)$$

- $SE(\hat{\beta}_j)$ is the standard error of the estimate.
- $t_{\alpha/2,\, n-p-1}$ is the critical value from the t-distribution with $n-p-1$ degrees of freedom.
- A 95% CI means: If we repeatedly sampled datasets and refit the model, 95% of such intervals would contain the true β_j.

1.17 OLS Violations

Ordinary Least Squares relies on several key assumptions. Violations of these assumptions can lead to unstable coefficients, incorrect inference, or inefficient predictions. Below are the main types of OLS assumption violations.

1.17.1 Multicollinearity

- Occurs when two or more explanatory variables are highly correlated.
- Multicollinearity increases the variance of coefficient estimates, making confidence intervals wider.
- The model becomes unstable: small changes in the data can cause large changes in coefficients.
- Diagnosis: Variance Inflation Factor (VIF) is the conventional tool.
- Warning signs:
  - Coefficient signs differ from theoretical expectations.
  - Adding or removing a feature causes coefficients to change dramatically.
  - Adding or deleting observations changes coefficients substantially.
- Remedies:
  - Combine correlated variables (feature engineering).
  - Remove or replace redundant features.
  - Use regularization (Ridge, Elastic Net).

1.17.2 Heteroskedasticity

- Refers to non-constant variance of the error term across observations.
- Residuals exhibit unequal scatter across the range of fitted values.
- Causes inefficient and biased standard errors, leading to unreliable hypothesis tests.
- Detection:
  - Residuals vs. fitted plot (visual pattern or "funnel shape").
  - Breusch–Pagan test (formal test).
- Remedies:
  - Transformations (log, square root).
  - Robust (heteroskedasticity-consistent) standard errors.
  - Weighted least squares.

1.17.3 Non-Normality of Errors

- Violates the assumption that residuals are normally distributed.

- Affects confidence intervals, p-values, and significance tests.
- Detection:
  - Q-Q plot.
  - Shapiro–Wilk, Kolmogorov–Smirnov, Anderson–Darling tests.
- Common sources:
  - Outliers.
  - Skewed or heavy-tailed distributions.
  - Omitted variables or incorrect functional form.
- Remedies:
  - Box–Cox or log transformations.
  - Remove or address outliers.
  - Respecify the model (modify functional form).

1.17.4 Serial Correlation (Autocorrelation)

- Occurs when error terms are correlated across time (common in time series data).
- Violates independence of errors, leading to underestimated standard errors.
- Detection:
  - Durbin–Watson test.
  - Plot of residuals over time.
- Remedies:
  - Include lagged variables.
  - Use time-series models (ARIMA, GLS).
  - Apply Newey–West robust standard errors.

1.18 Residual Diagnostics

Residual diagnostics help verify whether the OLS assumptions hold for a fitted regression model. These tools allow us to identify non-linearity, heteroskedasticity, non-normal errors, serial correlation, influential observations, and multicollinearity.

- Used to assess non-linearity and heteroskedasticity.
- Ideally, residuals should be randomly scattered around zero with no visible pattern.
- Systematic curves suggest non-linearity; changing spread suggests heteroskedasticity.

1.18.1 Q–Q Plot (Quantile–Quantile)

A Q–Q plot compares the quantiles of the residuals to those of a theoretical normal distribution.

- Points on a straight line indicate normal residuals.
- Curvature suggests skewness.
- Heavy tails appear as deviations at the ends of the plot.
- Outliers appear far from the reference line.

1.18.2 Shapiro–Wilk Test for Normality

- Tests whether residuals follow a normal distribution.
- Null hypothesis: residuals are normally distributed.
- Particularly powerful for small sample sizes ($n < 50$).

1.18.3 Kolmogorov–Smirnov Test for Normality

- Compares the empirical distribution of residuals to a theoretical normal CDF.
- Null hypothesis: residuals follow the normal distribution.
- Sensitive to differences in both shape and location.

1.18.4 Anderson–Darling Test for Normality

- A refinement of the K–S test with extra sensitivity to tail behavior.
- Null hypothesis: residuals are normally distributed.
- Strong at detecting heavy tails or deviations in extremes.

1.18.5 One-Sample t-test: Mean of Residuals = 0

- Tests whether the residuals have zero mean, as required under OLS assumptions.
- Null hypothesis: $\mathbb{E}[e_i] = 0$.

$$t = \frac{\bar{e}}{s_e/\sqrt{n}},$$

1.18.6 Leverage and Influence (Leverage Plot)

- Leverage measures how far an observation's predictor values are from the mean of the predictors.
- High-leverage points can strongly affect the regression surface.
- Even a point with a small residual can be influential if its leverage is high.

Cook's Distance:

- Measures the overall influence of an observation on the fitted regression coefficients.
- Rule of thumb: $D_i > 1$ indicates potentially influential observations.

1.19 Leverage

- Leverage measures how unusual a point's predictor values are.
- A point far away in feature space has high leverage.
- Hat matrix:
$$H = X(X^T X)^{-1} X^T$$
- The diagonal elements of the hat matrix, h_{ii}, are called leverage values.
- High leverage points correspond to observations with unusual predictor values.
- If
$$h_{ii} > \frac{2p}{n}$$
then the observation is considered a high leverage point.

1.20 Cook's Distance

- Cook's Distance measures how much the fitted model changes if an observation is removed.
- It combines residual magnitude and leverage:

$$D_i = \frac{e_i^2}{p \cdot MSE} \cdot \frac{h_{ii}}{(1 - h_{ii})^2}$$

- e_i = residual for observation i
- h_{ii} = leverage of observation i
- p = number of parameters (including intercept)
- $MSE = \dfrac{RSS}{n - p}$
- $D_i > 1$ may indicate a highly influential point.

1.20.1 Scale–Location Plot

- Also known as the Spread–Location plot.
- Plots $\sqrt{|e_i|}$ against fitted values to diagnose heteroskedasticity.
- A horizontal line with random scatter suggests constant variance.
- Systematic patterns indicate heteroskedasticity:
  - Upward trend: variance increases with fitted values (classic funnel shape).
  - Downward trend: variance decreases with fitted values.

1.20.2 Breusch–Pagan Test for Homoscedasticity

- Tests whether error variance is constant.
- Null hypothesis: homoscedastic errors.
- Alternative: heteroskedasticity dependent on predictors.

The auxiliary regression is:

$$\hat{\varepsilon}_i^2 = \gamma_0 + \gamma_1 x_{i1} + \cdots + \gamma_p x_{ip} + u_i$$

Let R^2_{aux} be the R^2 from this regression. The Breusch–Pagan statistic is:

$$W = nR^2_{\text{aux}}, \qquad W \sim \chi^2_p \text{ under the null.}$$

1.20.3 Durbin–Watson Test for Autocorrelation

- Detects first-order autocorrelation in residuals (commonly in time-series data).
- Null hypothesis: no first-order autocorrelation.

Correct formula:

$$d = \frac{\sum_{t=2}^{T}(e_t - e_{t-1})^2}{\sum_{t=1}^{T} e_t^2}.$$

- $d \approx 2$: no autocorrelation.
- $d < 1.5$: positive autocorrelation.
- $d > 2.5$: negative autocorrelation.

1.20.4 Variance Inflation Factor (VIF): Multicollinearity

$$VIF_j = \frac{1}{1 - R_j^2},$$

where R_j^2 is obtained from regressing predictor X_j on all other predictors.

- $VIF = 1$: no multicollinearity.
- $VIF > 5$: moderate multicollinearity concern.
- $VIF > 10$: severe multicollinearity.

1.21 Likelihood, AIC, and BIC in Linear Regression

Consider the multiple linear regression model:

$$y_i = x_i^\top w + \epsilon_i$$

where the error term is assumed to follow a Gaussian distribution:

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

This assumption implies:

$$y_i \mid x_i \sim \mathcal{N}(x_i^\top w, \sigma^2)$$

Thus, each target value is normally distributed around the regression prediction.

Likelihood Function For a single observation:

$$p(y_i \mid x_i, w, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^\top w)^2}{2\sigma^2}\right)$$

Assuming observations are independent, the likelihood of the entire dataset becomes:

$$L(w, \sigma^2) = \prod_{i=1}^{n} p(y_i \mid x_i, w, \sigma^2)$$

Substituting the Gaussian density:

$$L(w, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^\top w)^2}{2\sigma^2}\right)$$

Log-Likelihood Products are difficult to optimize, so we take logarithms.
Using:

$$\log(ab) = \log a + \log b$$

and:

$$\log(e^x) = x$$

we obtain:

$$\log L = \sum_{i=1}^{n} \log p(y_i \mid x_i, w, \sigma^2)$$

Expanding:

$$\log L = \sum_{i=1}^{n} \left[\log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{(y_i - x_i^\top w)^2}{2\sigma^2} \right]$$

Using:

$$\log \left(\frac{1}{\sqrt{a}} \right) = -\frac{1}{2} \log a$$

we get:

$$\log L = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - x_i^\top w)^2$$

Separating the logarithm:

$$\log(ab) = \log a + \log b$$

gives:

$$\boxed{\log L = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} RSS}$$

where:

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

is the Residual Sum of Squares.

Maximum Likelihood Estimate of Variance The maximum likelihood estimate of the variance is:

$$\boxed{\hat{\sigma}^2 = \frac{RSS}{n}}$$

Substituting this into the log-likelihood:

$$\log L = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \left(\frac{RSS}{n} \right) - \frac{RSS}{2(RSS/n)}$$

Simplifying the final term:

$$\frac{RSS}{2(RSS/n)} = \frac{n}{2}$$

Thus:

$$\log L = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log\left(\frac{RSS}{n}\right) - \frac{n}{2}$$

Factorizing:

$$\boxed{\log L = -\frac{n}{2}\left[\log(2\pi) + \log\left(\frac{RSS}{n}\right) + 1\right]}$$

Akaike Information Criterion (AIC) The Akaike Information Criterion is defined as:

$$\boxed{AIC = 2k - 2\log L}$$

where:

- $k =$ number of estimated parameters
- $\log L =$ maximized log-likelihood

Substituting the log-likelihood:

$$AIC = 2k + n\left[\log(2\pi) + \log\left(\frac{RSS}{n}\right) + 1\right]$$

Rearranging:

$$AIC = n\log\left(\frac{RSS}{n}\right) + 2k + n\log(2\pi) + n$$

Notice that:

$$n\log(2\pi) + n$$

is constant for all models trained on the same dataset.
Since AIC is used for comparing models, constant terms are usually ignored.
Thus the simplified practical formula becomes:

$$\boxed{AIC = n\log\left(\frac{RSS}{n}\right) + 2k}$$

Bayesian Information Criterion (BIC) The Bayesian Information Criterion is defined as:

$$\boxed{BIC = k\log(n) - 2\log L}$$

Substituting the Gaussian log-likelihood and removing constants gives:

$$\boxed{BIC = n\log\left(\frac{RSS}{n}\right) + k\log(n)}$$

Interpretation Both AIC and BIC balance:

1. Goodness of fit

   Lower RSS $\Rightarrow$ Higher likelihood

2. Model complexity

   More parameters $\Rightarrow$ Higher penalty

Thus:

- Adding predictors usually decreases RSS
- But adding too many predictors increases model complexity
- AIC and BIC penalize unnecessarily complex models

Difference Between AIC and BIC

$$AIC = 2k - 2\log L$$

$$BIC = k\log(n) - 2\log L$$

Since:

$$\log(n) > 2 \quad \text{for sufficiently large } n$$

BIC usually applies a stronger penalty than AIC and therefore tends to prefer simpler models.

2 Logistic Regression

- Discriminative probability classifier.
- Models the probability of a binary outcome as a smooth function of predictors, constrained between 0 and 1.

$$\begin{aligned} Y &\in \{0,1\} \\ P(Y=1) &= p \\ P(Y=0) &= 1-p \\ P(Y=y) &= p^y(1-p)^{1-y} \end{aligned}$$

- We start with Bernoulli pdf
- Write it in exponential form
- The coefficient multiply y must be the natural parameter
- so log-odds is not a design choice, it emerges.

Consider; $p^y(1-p)^{1-y}$, lets write it in exponential form;

$$\begin{aligned} p^y(1-p)^{1-y} &= \exp(y\log p + (1-y)\log 1-p) \\ &= \exp(y\log p + \log(1-p) - y\log 1-p) \\ &= \exp\left(y\log\frac{p}{1-p} + \log(1-p)\right) \end{aligned}$$

Canonical exponential form is as follows;

$$f(y|\theta) = \exp(y\theta - A(\theta))$$

So, based on our exponential form of Bernoulli, we have;

$$\begin{aligned} \theta &= \log\frac{p}{1-p} \\ -A(\theta) &= \log(1-p) \end{aligned}$$

So, $\log\frac{p}{1-p}$ is a natural parameter. Introducing Covariates : Conditional Bernoulli Once predictors exist, we are no longer modeling p, but;

$$p(x) = P(Y=1|X=x)$$

So, the natural parameter becomes conditional;

$$\theta(x) = \log\left(\frac{p(x)}{1-p(x)}\right)$$

We make one structural assumption; The natural parameter depends linearly on predictors;

$$\theta(x) = \beta_0 + \beta^T x$$

- $\theta \in (-\infty, +\infty)$
- Linear structure
- Linear natural parameter implies convex log-likelihood
- When we plug back the theta value into $p(x)$

2.1 Linking Functions

- Probit - Normal CDF
- Complementary log-log
- Cauchit
- Only logit is the canonical link for Bernoulli
- Canonical link $\rightarrow$ best statistical properties.

2.2 Parameter Estimation using MLE

For a single observation (x_i, y_i) with $y_i \in \{0, 1\}$, the Bernoulli probability mass function is:

$$P(Y_i = y_i \mid X_i = x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

$$p_i = P(Y_i = 1 \mid X_i = x_i)$$

Assuming **conditional independence** of observations given X, the likelihood for the full dataset $\{(x_i, y_i)\}_{i=1}^n$ is the product of individual likelihoods:

$$L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}$$

Here, the probabilities p_i are modeled using the logistic function:

$$p_i = \frac{1}{1 + \exp(-\eta_i)}$$
$$\eta_i = \beta_0 + \beta^T x_i$$

Since the log function is monotonic, maximising the likelihood is equivalent to maximising the log-likelihood.

$$\begin{aligned}\ell(\beta) &= \log L(\beta) \\ &= \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]\end{aligned}$$

This is the **log-likelihood** of logistic regression. In optimisation, it is common to convert a maximisation problem into a minimisation problem. We therefore define the **negative log-likelihood (NLL)**:

$$-\sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

The goal of logistic regression is;

$$\hat{\beta} = \arg\min_{\beta} \ \mathcal{L}(\beta)$$

- Logistic Regression chooses β so that the predicted probabilities $\sigma(\beta^T x_i)$ match the observed labels as well as possible.

The negative log-likelihood above is known in machine learning as the **binary cross-entropy loss** (or log loss). For a single observation, the loss is:

$$BCE(y_i, p_i) = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

For the full dataset;

$$BCE(\beta) = \sum_{i=1}^{n} BCE(y_i, p_i)$$

- This loss arises **directly from maximum likelihood estimation**, not as a heuristic.

- Thus, minimising binary cross-entropy is equivalent to **maximum likelihood estimation for a Bernoulli model with a logistic link**.

Thus, Logistic regression estimates parameters β by:

- Modeling $P(Y = 1 \mid X = x)$ via the logistic (sigmoid) function
- Writing the Bernoulli likelihood for observed labels
- Maximising the log-likelihood (or equivalently, minimising binary cross-entropy)
- Binary cross entropy loss comes directly from MLE.
- Logistic regression = maximise log-likelihood
- Equivalently = minimise BCE Loss
- The algorithm starts with random parameters β. The probabilities are computed. There are many algorithms which we can use for iterative optimisation (logistic regression has no closed form solution).
- If we take the gradient and set to zero, the equation will be nonlinear in β, because sigmoid function contains an exponential. No algebraic manipulation can isolate β. So, there is no analytic solution to that equation.
- The sigmoid introduces exponentials of β making the likelihood equations transcendental rather than linear or polynomial.

2.3 Interpretations of the coefficients

- β_1 - change in log-odds for 1 unit increase in x_1
- A one-unit increase in x_1 multiplies the odds of $y = 1$ by beta, holding other variables constant.

2.4 Key Logistic Assumptions

- Logit is linear in parameters

$$\log\left(\frac{p}{1-p}\right) = X\beta$$

- Observations are independent
- No severe multicollinearity
- No perfect separation

2.5 Odds, log-odds, odds ratio - Interpretations

$$\text{odds} = \frac{p}{1-p}$$

- Odds gives: success (p) is how many times as likely as failure.
- This value ranges from 0 to infinity, can't model linearly with unrestricted $X\beta$.
- When we take the log of odds, this ranges from $-\infty$ to $+\infty$.
- So $+$log odds means probability > 0.5 and -ve log odds means probability < 0.5.
- In logistic regression we assume: the log odds has a linear parameter structure with the predictors. This value can be modelled linearly with unrestricted $X\beta$.

  $$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots$$

  This ensures that we can get the probability between 0 and 1 (using sigmoid).

  $$\log\left(\frac{p}{1-p}\right) = -1.05 + 1.95x_1 - 0.98x_2$$

- If Odds Ratios of the parameters are (Taking the exponentials of the betas): 0.35, 7.0 and 0.37 respectively
- One unit increase in x_1 multiplies the odds of $y = 1$ by 7
- One unit increase in x_2 reduces odds by 63% $(1 - 0.37)$.
- 0.35 is the odds of $y = 1$ at $x_1 = x_2 = 0$

2.6 Logistic & Linear Regression Have the Same Gradient Form

It is interesting that the gradient updates for linear regression and logistic regression have the same form:

$$dw = \frac{1}{n} X^T(\hat{y} - y), \qquad db = \frac{1}{n}\sum(\hat{y} - y)$$

Even though the models and loss functions are different. The derivations below show why this happens.

Linear Regression Model:

$$\hat{y} = Xw + b$$

Loss function (Mean Squared Error):

$$L = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Gradient derivation

$$\frac{\partial L}{\partial w} = \frac{\partial}{\partial w}\left(\frac{1}{2n}\sum(\hat{y} - y)^2\right)$$

Applying the chain rule:

$$\frac{\partial L}{\partial w} = \frac{1}{n}\sum(\hat{y} - y)\frac{\partial \hat{y}}{\partial w}$$

Since

$$\hat{y} = Xw + b$$

$$\frac{\partial \hat{y}}{\partial w} = X$$

Therefore

$$\frac{\partial L}{\partial w} = \frac{1}{n} X^T(\hat{y} - y)$$

Similarly,

$$\frac{\partial L}{\partial b} = \frac{1}{n}\sum(\hat{y} - y)$$

Logistic Regression Model:

$$z = Xw + b$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (Binary Cross-Entropy):

$$L = -\frac{1}{n}\sum\left[y\log(\hat{y}) + (1 - y)\log(1 - \hat{y})\right]$$

Gradient derivation

Using the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

First compute

$$\frac{\partial L}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)$$

Sigmoid derivative:

$$\frac{\partial \hat{y}}{\partial z} = \sigma(z)(1-\sigma(z)) = \hat{y}(1-\hat{y})$$

Also,

$$\frac{\partial z}{\partial w} = X$$

Combining the terms:

$$\frac{\partial L}{\partial w} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)\hat{y}(1-\hat{y})X$$

Simplifying,

$$\frac{\partial L}{\partial w} = (\hat{y} - y)X$$

Averaging over all samples:

$$\frac{\partial L}{\partial w} = \frac{1}{n}X^T(\hat{y} - y)$$

Similarly,

$$\frac{\partial L}{\partial b} = \frac{1}{n}\sum(\hat{y} - y)$$

Although the **loss functions are different** (MSE vs cross-entropy), the special combination of the **sigmoid function and cross-entropy loss** causes the sigmoid derivative to cancel out during differentiation.
As a result, the gradient simplifies to the same form as linear regression:

$$dw = \frac{1}{n}X^T(\hat{y} - y)$$

The difference between the models lies in the definition of $\hat{y}$:

Linear regression: $\hat{y} = Xw + b$

Logistic regression: $\hat{y} = \sigma(Xw + b)$

2.7 Standard Errors of Logistic Regression Coefficients

- Coefficients are estimated using **Maximum Likelihood Estimation (MLE)**.
- Hessian / Fisher Information Matrix:
$$H = X^T W X$$
- Weight matrix:
$$W = \text{diag}(p_i(1-p_i))$$
- Covariance matrix of coefficients:
$$\text{Cov}(\hat{\beta}) = (X^T W X)^{-1}$$
- Standard error of coefficient $\hat{\beta}_j$ is given by:
$$SE(\hat{\beta}_j) = \sqrt{[(X^T W X)^{-1}]_{jj}}$$
- Standard errors quantify uncertainty in coefficient estimates.
- Used in Wald Test, confidence intervals, and p-value computation.

2.8 Hypothesis Testing & Likelihood Ratio Test

- **Wald Test**: Most common coefficient significance test.

$$z = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Under H_0:

$$z \sim N(0,1)$$

Equivalent chi-square form:

$$z^2 \sim \chi_1^2$$

- **Likelihood Ratio Test** for overall model significance:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$$

- In this test, we compare the null model (intercept only) with the full model.
- Likelihood Ratio Statistic is given by:

$$LR = -2\left[\log L_0 - \log L_1\right]$$

- Under H_0:

$$LR \sim \chi_k^2$$

where k is the number of predictors added.

2.9 Leverage

- Leverage measures how unusual an observation's predictor values are.
- Leverage matrix:

$$H = W^{1/2} X (X^T W X)^{-1} X^T W^{1/2}$$

- Leverage values are diagonal elements:

$$h_i = H_{ii}$$

- High leverage points can strongly influence model fitting.

2.10 Cook's Distance

- **Cook's Distance**:

$$D_i = \frac{r_i^2 h_i}{p(1-h_i)^2}$$

- r_i = standardized residual
- h_i = leverage
- p = number of parameters
- Large Cook's Distance indicates influential observations.
- Common threshold:

$$D_i > \frac{4}{n}$$

where n is the number of observations.

2.11 Goodness of Fit in Logistic Regression

- Goodness of fit measures how well the model explains observed outcomes.
- Residual Deviance is the primary likelihood-based fit metric:

$$D = -2(\log L_{\text{model}} - \log L_{\text{saturated}})$$

- Lower deviance indicates better model fit.
- **Hosmer–Lemeshow Test** compares observed vs predicted probabilities across groups.
- Hypotheses:

$$H_0 : \text{Model fits data well}$$

$$H_1 : \text{Model does not fit data well}$$

- Hosmer–Lemeshow test statistic:

$$HL = \sum_{g=1}^{G} \left(\frac{(O_{1g} - E_{1g})^2}{E_{1g}} + \frac{(O_{0g} - E_{0g})^2}{E_{0g}} \right)$$

- Test statistic approximately follows:

$$HL \sim \chi^2_{g-2}$$

  where g is the number of groups.
- Large p-value (> 0.05) indicates good fit, whereas small p-value (< 0.05) indicates poor fit.

2.12 McFadden R^2 in Logistic Regression

- Logistic regression has no true variance-based R^2.
- McFadden R^2 is a likelihood-based pseudo R^2 measure.

$$R^2_{\text{McFadden}} = 1 - \frac{\log L_{\text{full}}}{\log L_{\text{null}}}$$

- $\log L_{\text{full}}$ = log-likelihood of fitted model
- $\log L_{\text{null}}$ = log-likelihood of intercept-only model
- Larger values indicate better model fit.
- Typical good values lie between $0.2 - 0.4$.
- Uses the same likelihood principle as the Likelihood Ratio Test.

2.13 Box–Tidwell Test (Linearity of Log-Odds)

- Used to check whether predictors have a linear relationship with log-odds in logistic regression.
- For predictor x, create interaction term:

$$x \log(x)$$

- Extended logistic regression model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x + \beta_2 x \log(x)$$

- Hypotheses:

$$H_0 : \beta_2 = 0$$

$$H_1 : \beta_2 \neq 0$$

- Applied separately for each continuous predictor.

3 Classification Evaluation Metrics

Evaluation metrics help measure how well a classification model performs. Many of these metrics are derived from the confusion matrix, which summarizes prediction outcomes.

3.1 Confusion Matrix

For a binary classification problem:

- TP (True Positive): Model correctly predicts the positive class.
- TN (True Negative): Model correctly predicts the negative class.
- FP (False Positive): Model predicts positive but the true label is negative.
- FN (False Negative): Model predicts negative but the true label is positive.

3.2 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy measures the overall fraction of correct predictions. However, it can be misleading for imbalanced datasets.

3.3 Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision answers the question:

Of all the instances predicted as positive, how many were actually positive?

High precision means the model produces few false positives.

3.4 Recall (Sensitivity / True Positive Rate)

$$\text{Recall (Sensitivity/TPR)} = \frac{TP}{TP + FN}$$

Recall answers:

Of all actual positive cases, how many did the model correctly identify?

High recall means the model misses few positive cases.

3.5 Specificity (True Negative Rate)

$$\text{TNR (Specificity)} = \frac{TN}{TN + FP}$$

Specificity measures how well the model identifies negative cases.

Of all actual negatives, how many did the model correctly classify?

3.6 False Positive and False Negative Rates

$$\text{FPR} = \frac{FP}{FP + TN}$$

$$\text{FNR} = \frac{FN}{FN + TP}$$

- FPR measures how often negative samples are incorrectly classified as positive.
- FNR measures how often positive samples are missed by the model.

3.7 F-Score

Precision and recall often conflict with each other. The **F-score** combines them into a single metric.

$$F_\beta = \frac{(1+\beta^2)\, p\, r}{(\beta^2 p) + r}$$

where

- $p =$ precision
- $r =$ recall

Special case:

$$F_1 = \frac{2pr}{p+r}$$

- $\beta > 1$ emphasizes **recall**
- $\beta < 1$ emphasizes **precision**
- $\beta = 1$ balances both equally

3.8 ROC Curve

Many classifiers output a **probability score**. By changing the classification threshold, we obtain different values of TPR and FPR.

The **ROC (Receiver Operating Characteristic) curve** plots:

True Positive Rate vs False Positive Rate

for all possible thresholds.

A good classifier produces a curve that rises quickly toward the top-left corner.

3.9 AUC (Area Under the ROC Curve)

AUC summarizes the ROC curve into a single number.

- Range: $0 \leq AUC \leq 1$
- $AUC = 0.5 \rightarrow$ random guessing
- $AUC = 1 \rightarrow$ perfect classifier

Interpretation:

AUC represents the probability that a randomly chosen positive instance receives a higher score than a randomly chosen negative instance.

Example:

- AUC $= 0.85$ means a random positive scores higher than a random negative **85% of the time**.

3.10 Gini Coefficient

The Gini coefficient is a measure of the **separation power** of a classification model and is directly derived from the Area Under the ROC Curve (AUC).

$$\text{Gini} = 2 \cdot \text{AUC} - 1$$

AUC measures the probability that a randomly chosen positive instance is ranked higher than a randomly chosen negative instance. However, a completely random classifier already has

$$\text{AUC} = 0.5$$

Since we often want to measure **how much better a model performs compared to random ranking**, we remove this baseline and rescale:

$$\text{Gini} = 2(\text{AUC} - 0.5) = 2 \cdot \text{AUC} - 1$$

Thus, the Gini coefficient represents the normalized improvement of the model over random classification.

- Gini $= 0 \rightarrow$ random model
- Gini $= 1 \rightarrow$ perfect model

The Gini coefficient is widely used in credit risk modeling and scoring systems.

Summary: Gini measures how much better a model ranks positives above negatives compared to a random model.

3.11 Multi-Class Confusion Matrix

For multi-class classification, a confusion matrix shows how predictions are distributed across classes.

	Predicted			
Actual	Apple	Orange	Mango	Total
Apple	7	8	9	24
Orange	1	2	3	6
Mango	3	2	1	6
Total	11	12	13	36

For each class, we treat it as the **positive class** and all other classes as **negative**. This converts the multi-class problem into multiple binary evaluation problems.

	TP	FP	FN	TN
Apple	7	17	4	8
Orange	2	4	10	20
Mango	1	5	12	18
Total	10	26	26	46

3.12 Micro, Macro, and Weighted Averages

When evaluating multi-class models, metrics can be aggregated in different ways.

- **Macro Average**
  Compute the metric independently for each class and then take the average.
  - Treats all classes equally
  - Useful when classes are balanced
- **Micro Average**
  Aggregate TP, FP, FN across all classes and then compute the metric.
  - Gives more weight to larger classes
- **Weighted Average**
  Compute the metric per class but weight each class by its frequency.
  - Useful when class imbalance exists

3.13 KS Statistic (Kolmogorov–Smirnov)

The **KS statistic** measures the maximum separation between the score distributions of positive and negative classes. It is commonly used in **credit scoring and risk modeling**.

$$CDF_{\text{Pos}} = \% \text{ of positives with score} \leq p$$

$$CDF_{\text{Neg}} = \% \text{ of negatives with score} \leq p$$

$$KS = \max_{p} |CDF_{\text{Pos}} - CDF_{\text{Neg}}|$$

- KS measures how well the model separates positive and negative score distributions.
- Range: 0 to 1.
- Higher KS indicates better class separation.

Typical interpretation:

- KS $< 0.2 \rightarrow$ weak model
- KS $0.2 - 0.4 \rightarrow$ moderate model
- KS $> 0.4 \rightarrow$ strong separation

3.14 Focal Loss

$$L = -(1 - p_t)^{\gamma} \log(p_t)$$

- Extension of cross-entropy designed for class imbalance.
- p_t is the predicted probability for the true class.
- Down-weights easy examples; focuses learning on hard misclassified samples.
- γ (focusing parameter) controls strength of modulation.
- Common in object detection tasks with extreme imbalance.

4 Bias–Variance Tradeoff

4.1 Core Idea

The bias–variance tradeoff describes how model flexibility affects prediction accuracy and generalization.

- More flexible models fit training data better (lower bias) but are more sensitive to data (higher variance).
- Less flexible models are more stable (lower variance) but may underfit (higher bias).

4.2 Definitions

Assume a data-generating process $y = f(x) + \epsilon$, where ϵ has mean 0 and variance σ^2. A learned model $\hat{f}(x; D)$ depends on the random training set D.

$$\text{Bias}_D\left[\hat{f}(x; D)\right] = \mathbb{E}_D\left[\hat{f}(x; D)\right] - f(x)$$

Bias: Difference between the true function and the expected prediction of the model across datasets.

$$\text{Var}_D\left[\hat{f}(x; D)\right] = \mathbb{E}_D\left[\left(\hat{f}(x; D) - \mathbb{E}_D\left[\hat{f}(x; D)\right]\right)^2\right]$$

Variance: How much the prediction varies across different datasets drawn from the same population.

4.3 Bias–Variance Decomposition

At a fixed point x, the expected prediction error can be decomposed as:

$$\text{Error}(x) = \underbrace{\text{Bias}^2(x)}_{\text{systematic error}} + \underbrace{\text{Variance}(x)}_{\text{sensitivity to data}} + \underbrace{\text{Noise}}_{\text{irreducible}}$$

4.4 Trade-off:

- When you try to reduce bias, the variance usually increases.
- When you try to reduce variance, the bias usually increases.
- In most real-world models, you cannot minimize both simultaneously.
- σ^2 is irreducible noise.

- Bias and variance are defined by averaging over *datasets*, not test points.
- Bagging methods reduces the variance.
- Boosting methods reduces the bias.

5 Isolation Forest

- Designed specifically for anomaly detection
- Anomalies are few and different, and can therefore be isolated with fewer random splits than normal points.
- Isolation Forest does not rely on distance or density estimation. It isolates points using random partitions.
- Imagine you want to isolate a point by repeatedly splitting the feature space using random rules (like choosing a random dimension, then a random threshold).
- Normal points exist in dense clusters $\rightarrow$ they require many random splits to isolate.
- Anomalies exist in sparse regions $\rightarrow$ they require fewer splits to isolate.

To build one tree:

- Randomly sample ϕ points from the dataset (ϕ is the sub-sample size, often 256).
- If the node has 1 point, or the depth has reached max height then stop splitting.
- Else, Randomly select a feature x_j and randomly choose a split value between its min and max.
- Partition the data into left and right subsets.
- Recursively repeat.
- This grows a random binary tree
- Repeat the tree-building process t times.
- For each point, go through every tree and measure path length (number of edges from root to the leaf)

Let t be the number of trees and ψ be the subsample size used to build each tree.

Path length in a single tree

For a data point x, let $h_i(x)$ denote the path length in the i^{th} tree.

- If the traversal ends in a pure leaf (only one sample):

$$h_i(x) = \text{depth}(x)$$

- If it ends in a leaf containing $z > 1$ samples:

$$h_i(x) = \text{depth}(x) + c(z)$$

Here, $c(z)$ is the expected additional path length required to isolate a point among z samples.

$$c(z) = 2H_{z-1} - \frac{2(z-1)}{z}$$

where the harmonic number is defined as:

$$H_n = \sum_{k=1}^{n} \frac{1}{k}$$

Average path length across trees

$$\bar{h}(x) = \frac{1}{t} \sum_{i=1}^{t} h_i(x)$$

Normalization constant

The normalization uses the expected path length of an unsuccessful search in a binary tree of size ψ:

$$c(\psi) = 2H_{\psi-1} - \frac{2(\psi-1)}{\psi}$$

- $c(z)$: "how much more depth is missing?"
- $c(\psi)$: "what depth should a normal point have?"

Anomaly score

$$s(x) = 2^{-\frac{\bar{h}(x)}{c(\psi)}}$$

Interpretation

- $s(x) \approx 1; \Rightarrow;$ strong anomaly (short paths)
- $s(x) \approx 0.5; \Rightarrow;$ typical/normal points
- $s(x) \approx 0; \Rightarrow;$ highly normal (long paths)

5.1 subsampling in Isolation Forest

In a small random subsample:

- Ensures anomalies are rare in each sample
- Normal points are fewer
- Anomalies usually appear alone
- A few random splits isolate them quickly
- So anomalies get short path lengths, which is the core idea.

5.2 Hyperparameters

We summarize the key hyperparameters of the Isolation Forest model as implemented in `scikit-learn`.

- **n estimators**: This controls how many isolation trees are built in the ensemble. Increasing this value generally improves the stability and reliability of the anomaly scores, but also increases computation time.
- **max samples**: This determines how many data points are randomly selected to train each tree. By default, a small subset is used rather than the full dataset. This subsampling is important because isolation works more effectively and efficiently on smaller samples.
- **contamination**: This specifies the expected proportion of anomalies in the dataset. It does not influence how the model learns patterns. Instead, it is used after training to decide which data points should be labeled as anomalies based on their scores.
- **max features**: This determines how many features are used when building each tree. Instead of using all features, the model can randomly select a subset for each tree. This increases diversity among trees and can improve performance, especially when dealing with high-dimensional data.
- **bootstrap**: This indicates whether each tree is trained on a sample drawn with replacement. When enabled, some data points may appear multiple times in the same sample.
- **random state**: This controls the randomness involved in sampling data and selecting features, ensuring that results can be reproduced.
- **n jobs**: This specifies how many processor cores are used to train the model in parallel, which can significantly speed up computation.

6 K-Nearest Neighbors (KNN)

6.1 Core Idea

- KNN is a **non-parametric**, **instance-based**, **lazy learning** algorithm.
- **Non-parametric**: there is no fixed parameter vector to learn; the model is defined by the training data, the distance metric, and the value of k.
- **Instance-based**: predictions are made using the training points directly.
- **Lazy**: there is no training phase; all computation happens at prediction time.
- KNN is a **local averaging** estimator: it predicts from nearby points only.

Given a query point x, compute distances $d(x, x_i)$ to all training points, select the k nearest neighbors $\mathcal{N}_k(x)$, and predict using only those points:

$$\hat{f}(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i$$

$$\hat{y}(x) = \arg\max_c \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c)$$

- Regression: average the neighbor targets.
- Classification: majority vote among neighbor labels.

6.2 Choosing k

- Small $k \to$ very local, high variance, sensitive to noise.
- Large $k \to$ heavy averaging, high bias, over-smoothing.
- k is typically selected using cross-validation.

6.3 Distance Metrics

- Euclidean distance is common but assumes features are on comparable scales and uncorrelated.
- **Mahalanobis distance** accounts for feature scale and correlation:

$$d_M(x, z) = \sqrt{(x - z)^T \Sigma^{-1} (x - z)}$$

- It measures how many standard deviations apart two points are in the data's natural variation.
- If features are whitened (variance $= 1$ and no correlation), Mahalanobis reduces to Euclidean distance.
- **Feature scaling is mandatory**. Unscaled features distort distances and dominate neighbor selection.
- KNN assumes **local smoothness**: nearby points should have similar outputs.
- In high dimensions, distances become less informative (curse of dimensionality), which can degrade performance.
- Mixed data types and correlated features require careful metric selection or preprocessing.

7 Naive Bayes

7.1 Core Idea

Given features $X = (x_1, x_2, \ldots, x_n)$ and class $Y \in \{C_1, C_2, \ldots, C_k\}$, we compute the posterior $P(Y = C_k \mid X)$ and choose the class with the highest probability.

- Naive Bayes is a **probabilistic** and **generative** classifier (like GMM in spirit).
- It models $P(X \mid Y)$ and $P(Y)$, then uses Bayes' rule to infer $P(Y \mid X)$.

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}$$

- $P(Y)$: prior (class prevalence)
- $P(X \mid Y)$: likelihood (feature distribution within class)
- $P(X)$: evidence (normalization, same across classes)

$$P(Y \mid X) \propto P(X \mid Y)P(Y)$$

7.2 Naive Conditional Independence

The joint likelihood grows exponentially with the number of features:

$$P(X \mid Y) = P(x_1, x_2, \ldots, x_n \mid Y)$$

Naive Bayes assumes conditional independence of features given the class:

$$P(x_1, x_2, \ldots, x_n \mid Y) = \prod_{i=1}^{n} P(x_i \mid Y)$$

So the posterior becomes:

$$P(Y \mid X) \propto P(Y) \prod_{i=1}^{n} P(x_i \mid Y)$$

To avoid underflow, we work in log space:

$$\hat{Y} = \arg\max_{Y} \left[\log P(Y) + \sum_{i=1}^{n} \log P(x_i \mid Y) \right]$$

7.3 Estimating Priors

$$P(Y = C_k) = \frac{\text{Number of samples in class } C_k}{\text{Total number of samples}}$$

7.4 Common Likelihood Models

Gaussian Naive Bayes (continuous features)

$$x_i \mid Y = C_k \sim \mathcal{N}(\mu_{ik}, \sigma_{ik}^2)$$

$$P(x_i \mid Y = C_k) = \frac{1}{\sqrt{2\pi\sigma_{ik}^2}} \exp\left(-\frac{(x_i - \mu_{ik})^2}{2\sigma_{ik}^2} \right)$$

- Each feature is modeled with a class-conditional Gaussian.

Bernoulli Naive Bayes (binary features)

$$P(x_i \mid Y) = p_i^{x_i} (1 - p_i)^{1 - x_i}$$

- Use when features represent presence/absence.

Multinomial Naive Bayes (counts, text)

$$P(x_i \mid Y) = \frac{\text{count of word } i \text{ in class } Y + \alpha}{\text{total words in class } Y + \alpha V}$$

- Suitable for bag-of-words text features and count data.
- Laplace smoothing adds a pseudocount α to prevent zero probabilities.

Example: pet preferences (multinomial counts)

	Cat	Dog	Rabbit	Hamster	Fish	Total
Class 1	18	20	6	4	2	50
Class 2	15	15	10	5	5	50
Class 3	17	18	8	4	3	50

Each row can be viewed as a random draw from a multinomial distribution $M_5(n = 50; p_1, p_2, \ldots, p_5)$.
The multinomial PMF is:

$$X = (x_1, x_2, \ldots, x_V), \qquad \sum_i x_i = N$$

$$p(X = x \mid Y = C_k) = \frac{N!}{\prod_i x_i!} \prod_i \theta_{ik}^{x_i}$$

- The combinatorial term $\frac{N!}{\prod_i x_i!}$ depends only on the observed document, so it cancels in the class argmax.
- Classification uses:

$$\hat{y} = \arg\max_k \left(\prod_i \theta_{ik}^{x_i} P(C_k) \right)$$

- The independence assumption is usually false in real data (words in a sentence, pixels in an image, and medical symptoms are often correlated), but Naive Bayes often performs well in practice.
- Different feature types can be mixed by choosing an appropriate likelihood model per feature.

8 Support Vector Machines (SVM)

Support Vector Machine is a supervised learning algorithm for classification, regression, and outlier detection. It finds the decision boundary that maximizes the margin between classes.

- Maximizing margin improves generalization.
- Convex optimization gives a global optimum.
- Kernels allow non-linear decision boundaries.

$$w \cdot x + b = 0$$

$$\hat{y} = \begin{cases} +1, & \text{if } w \cdot x + b \geq 0 \\ -1, & \text{if } w \cdot x + b < 0 \end{cases}$$

If data are linearly separable, there are infinitely many separating hyperplanes. SVM selects the one with the largest margin.

- Margin hyperplanes: $w \cdot x + b = +1$ and $w \cdot x + b = -1$.
- All points satisfy: $y_i(w \cdot x_i + b) \geq 1$.
- Margin width: $\frac{2}{\|w\|}$.

8.1 Hard-Margin Optimization

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{subject to} \quad & y_i(w \cdot x_i + b) \geq 1, \quad i = 1, \ldots, n \end{aligned}$$

8.2 Soft-Margin SVM

When classes overlap, introduce slack variables ξ_i to allow controlled violations:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

- C controls penalty strength: larger C penalizes violations more.
- $0 \leq \xi_i < 1$: inside margin but correctly classified.
- $\xi_i = 1$: on the decision boundary.
- $\xi_i > 1$: misclassified.

8.3 Lagrangian and Support Vectors

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 + \sum_{i=1}^{n} \alpha_i \left(1 - y_i(w \cdot x_i + b)\right)$$

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0, \qquad \frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0$$

- Points with $\alpha_i > 0$ are **support vectors**.
- For soft margin, $0 \leq \alpha_i \leq C$.

8.4 Decision Function

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i (x_i \cdot x) + b$$

8.5 Hinge Loss

$$L = \max(0, 1 - y \cdot f(x))$$

- Used in margin-based classifiers (e.g., SVMs); enforces a decision boundary with a safety margin.
- Targets are $y \in -1, +1$; predictions are raw scores, not probabilities.
- Zero loss when the sample is correctly classified with sufficient margin.
- Linear penalty when the margin condition is violated.
- Encourages robustness by focusing only on "difficult" or misclassified points.

8.6 Primal & Dual Form of SVM

8.6.1 Primal Form

Given a training dataset $\{(x_i, y_i)\}_{i=1}^{n}$ with $y_i \in \{-1, +1\}$, the primal form of the (soft-margin) SVM is:

$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

subject to:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Interpretation:

- $\frac{1}{2}\|w\|^2$ controls margin maximization (smaller norm $\Rightarrow$ larger margin)
- ξ_i are slack variables allowing violations
- $C > 0$ balances margin size vs classification error

Geometric Insight: The primal directly searches for a hyperplane $w^\top x + b = 0$ that maximizes the margin while penalizing misclassification.

8.6.2 Dual Form

Introducing Lagrange multipliers $\alpha_i \geq 0$, the dual problem becomes:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^\top x_j)$$

subject to:

$$\sum_{i=1}^{n} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

Recovering Primal Variables:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

The bias b can be computed using any support vector satisfying $0 < \alpha_i < C$.

Key Observation: Only points with $\alpha_i > 0$ contribute to w. These are called **support vectors**.

8.6.3 Difference Between Primal and Dual

- **Optimization Variables:**
  - Primal: w, b, ξ
  - Dual: α_i
- **Dependence on Data:**
  - Primal: depends directly on feature vectors x_i
  - Dual: depends only on pairwise inner products $x_i^\top x_j$
- **Sparsity:**
  - Primal: all data points contribute
  - Dual: only support vectors (non-zero α_i) contribute
- **Dimensionality:**
  - Primal: depends on feature dimension d
  - Dual: depends on number of samples n

8.6.4 When to Use Which?

- Use **Primal** when:
  - Number of features d is small
  - Number of samples n is large
  - A direct optimization over w is computationally efficient
- Use **Dual** when:
  - Number of samples n is small to moderate
  - Solution sparsity is desired
  - Only a subset of points is expected to define the boundary

There is no universally better formulation; both are equivalent via convex duality.

- Primal is often more efficient for large-scale linear problems
- Dual is advantageous when the solution is sparse

In practice, the choice depends on the relationship between n (samples) and d (features), and computational considerations.

The dual is derived from the primal using the Lagrangian:

$$\mathcal{L}(w, b, \xi, \alpha, \mu) = \frac{1}{2}\|w\|^2 + C\sum \xi_i - \sum \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum \mu_i \xi_i$$

Applying KKT conditions:

- Stationarity:

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \Rightarrow w = \sum \alpha_i y_i x_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum \alpha_i y_i = 0$$

- Complementary slackness:

$$\alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] = 0$$

These conditions explain why only boundary points (support vectors) have non-zero α_i.

- Primal: directly finds the separating hyperplane
- Dual: determines which data points define the hyperplane
- Support vectors are the critical elements shaping the decision boundary

8.7 Kernel Trick

Map inputs with $\phi(x)$ into a higher-dimensional space:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i (\phi(x_i) \cdot \phi(x)) + b$$

Use a kernel $K(x_i, x) = \phi(x_i) \cdot \phi(x)$ to avoid explicit mapping:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b$$

- Kernels enable non-linear boundaries while keeping computations tractable.
- The Gaussian (RBF) kernel corresponds to an infinite-dimensional feature space.

Consider $x = (x_1, x_2, x_3)$ and a mapping $\phi(x)$ that creates all pairwise products:

$$\phi(x) = (x_1x_1, x_1x_2, x_1x_3, x_2x_1, x_2x_2, x_2x_3, x_3x_1, x_3x_2, x_3x_3)$$

This is a 3D $\rightarrow$ 9D mapping. The kernel computes the dot product in that 9D space without explicitly building $\phi(x)$:

$$K(x, y) = (x \cdot y)^2$$

- Same result as $\langle \phi(x), \phi(y) \rangle$, but far cheaper to compute.
- Kernels act as similarity measures in feature space.

9 Decision Tree Algorithm

Consider a dataset with N examples, with each feature $x = (x_1, x_2, \ldots, x_d)$ and output labels $y \in \{1, 2, \ldots, K\}$ (Classification) and $y \in \mathcal{R}$ (regression). The goal is to learn a function $f(x)$.

Decision tree partitions the input space into rectangular regions and assigns a constant prediction to each region. Each split cuts the space into two (or more) parts. After many splits we will be having small regions. Inside each region, the model predicts ; the majority class for classification and mean value for regression.

How does a tree decide where to split? A good split makes child nodes more 'pure'. For classification we want the child node contains mostly one class and for regression we want values close to each other inside a child node. So to do this we need a measure of impurity.

There are different choices for Impurity measures;

$$\text{Gini} = 1 - \sum_{k=1}^{K} p_k^2$$

- Gini $= 0$ when we have perfectly pure node
- Gini is max when classes are evenly mixed - uniform distribution

$$\text{Entropy} = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- If all points belong to one class $\rightarrow$ No Uncertainty
- If classes are evenly mixed $\rightarrow$ High Uncertainty

$$\text{Impurity after split} = \frac{n_L}{n} I_L + \frac{n_R}{n} I_R$$

- n_L, n_R : samples in left/right child
- I_L, I_R : impurities of children

- We choose a split that minimizes this quantity.

In Regression trees we the criteria is to minimize the variance/squared error;

$$\text{MSE for a split} = \sum_{i \in L}(y_i - \bar{y}_L)^2 + \sum_{i \in R}(y_i - \bar{y}_R)^2$$

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- Each leaf predicts a constant
- Best Constant = mean
- Good split = groups with low spread

We choose the best(feature, threshold) pair that yields max impurity reduction.

9.1 CART Algorithm

CART stands for classification and regression trees - it's the algorithm scikit-learn implements in both `DecisionTreeClassifier` and `DecisionTreeRegressor`

- Uses binary splits, even for nominal data
- For classification, impurity is measured by Gini/Gain
- For regression, impurity is MSE (Mean Squared Error)
- Always seeks the best split over candidate thresholds
- In CART algorithm we have pre-pruning only (usually)

This is not ID3 or C4.5 exactly - CART is similar to C4.5 but specialized for binary splits and regression as well.

For continuous data we use finite splits. The training dataset only has a finite number of values per feature. So the algorithm;

- Sorts the feature values seen at the current node
- Consider candidate thresholds only between adjacent sorted values.

$$t_i = \frac{v_i + v_{i+1}}{2}$$

- Evaluates each threshold by computing impurity before & after split
- Search is finite and efficient

Because scikit-learn cannot natively split categorical variables, practitioners encode them first;

- One-hot encoding : turns a categorical variable with many levels into binary columns
- Ordinal Encoding - sometimes okay for ordinal categories (like low-medium-high) but be careful if order isn't meaningful.
- This is why libraries like LightGBM or CatBoost add special categorical handling - but scikit-learn doesn't yet.

9.2 Stopping Criteria

If you let the tree grow ; it can memorize the data $\rightarrow$ Zero training error $\rightarrow$ Terrible generalization. So we have some common stopping rules :

- max depth
- min samples split
- min samples leaf
- min impurity decrease

Each split increases model flexibility

$$O(d * N \log N)$$

d is the feature dimension, N is the total number of samples. In prediction we have 0(depth) of complexity.

9.3 Merits of Decision Trees

- Interpretable model

- Handles Nonlinearity
- No feature Scaling
- Handles mixed data types (Not in sklearn)
- Fast Inference

9.4 Demerits of Decision Trees

- High Variance (Highly flexible Model)
- Unstable (Small data change → Different Tree)
- Greedy suboptimal splits
- Axis-Aligned Splits Only
- Poor Extrapolation

9.5 Properties of Decision Trees

- Trees are piecewise constant → Within each region, prediction does not change
- Trees approximate functions using rectangles → Each Rm is an axis-aligned box
- Trees cannot extrapolate → Outside training regions, prediction is still a constant
- Trees are not smooth → No continuity, no gradients

9.6 Pre-Pruning Decision Trees

We want a prediction function $\hat{f_T}(x)$. For a tree T, define training error as ;

$$R(T) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, \hat{f}_T(x_i))$$

For classification → Misclassification loss / Gini Proxy

For regression → Squared error

This is how well the tree fits training data.

$$\min_T R(T) \quad \text{subject to constraints}$$

These constraints prevent splits from ever being considered. Pre-pruning restricts the hypothesis space before optimization. It does not change the loss function - it changes what trees are allowed. Because decision trees are greedy; A split that looks weak now might enable strong future splits. Pre-pruning may block those future improvements.

9.7 Cost - Complexity Pruning (Post-Pruning)

CART defines a regularised objective;

$$R_\alpha(T) = R(T) + \alpha|T|$$

$R(T) =$ Training Error (data fit)

$|T| =$ Number of leaf nodes (Model complexity)

$\alpha \geq 0 =$ Complexity penalty (How much we penalize complexity)

The optimal tree is the one that minimizes this quantity.

- Different alpha values prefer different tree sizes
- Small alpha → complex trees allowed
- Large alpha → Simple trees preferred.

This is regularisation, exactly like L1/L2 - but for trees.

$$T^*(\alpha) = \arg\min_{T \subseteq T_0} R_\alpha(T)$$

Among all subtrees of the full tree, we choose the one that minimizes penalized risk. CART does not try all subtrees (too many). Instead it proves that only a finite sequence of candidate trees can be optimal. These trees form a nested sequence

Each step removes the weakest link; What is a weakest link?

For an internal node t;

$$\alpha_t = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

T_t is the entire subtree rooted at t. So t is the root of that subtree. T_t includes - node t, all its descendants and all leaf nodes under it. and Now $|T_t|$ is the number of lead nodes in the subtree T_t. Why leaves? Each leaf corresponds to one region R_m. Each leaf adds one constant prediction. Leaves measure model complexity.

- Pruning at t reduces the model complexity by $|T_t| - 1$ leaves.
- $R(T_t)$ - Training error of the entire subtree. Sum of losses over all leaves under t.
- $R(t)$ - Training error if we collapse the subtree into a single leaf.
- The numerator difference will increase in training error caused by pruning this subtree.
- So α_t measures error increase per unit of complexity reduction.

$$\alpha_t = \frac{\text{increase in training error}}{\text{number of leaves removed}}$$

CART wants to prune, subtrees that don't reduce error much but add a lot of complexity. So it removes the subtree with smallest α_t

CART does not choose alpha upfront. Alpha emerges from the tree structure itself.

- The tree has a finite number of internal nodes
- CART computes a finite set of critical alpha values.
- The smallest alpha corresponds to the weakest link
- That subtree is pruned first
- CART may prune multiple internal nodes at the same step if they have the same weakest alpha.
- The pruning step will be over till it reaches the single node tree.
- Each pruning step produces one alpha breakpoint

In scikit-learn tree cost complexity pruning path returns sorted alpha breakpoints and the impurities corresponding subtree errors. Each ccp-alpha prunes the tree up to that step.

CART computes a critical Alpha value for each internal node. At each pruning step, it collapses all internal nodes with the smallest alpha, then recomputes these values. This process continues until the tree reduces to a single leaf, producing a finite, nested sequence of trees indexed by increasing alpha

We can select T^* with the minimum validation error. Only T* goes to production.

Alpha - average increase in error per leaf that would be removed if we prune that subtree.

So if the alpha is high then we cannot prune that part of the tree ;

10 Ensemble Methods

Ensemble methods combine multiple models - base learner to produce a single stronger model.

- Different models make different mistakes and aggregation smooths them out
- Base learners are usually simple (decision trees, linear models).

10.1 Bagging

- Bagging focuses on variance reduction
- Multiple models are trained independently
- Each model sees a different bootstrap sample (sampling with replacement)
- Predictions are combined by averaging or majority vote.
- Works best for high variance models (decision trees)

10.2 Boosting

- Focuses on bias reduction
- Models are trained sequentially
- Each new model focuses on previous mistakes

- Predictions are combined using weighted sums

10.3 Stacking

- Combines different types of models
- Multiple diverse base models are trained in parallel
- Their predictions are used as features for a meta-model
- The meta model learns how to weight or combine base predictions.
- logistic regression on top of RF + SVM + GBM

10.4 Voting

Voting combines multiple models using simple aggregation rules

- Hard voting : majority class vote
- Soft voting : Averaging predicted probabilities
- Models are usually diverse
- High variance, unstable model → **Bagging / Random Forest**
- High bias, underfitting → **Boosting**
- Very different model types → **Stacking**
- Simple ensemble baseline → **Voting**
- Categorical-heavy data → **CatBoost**
- Massive dataset, speed matters → **LightGBM**

11 Random Forest

Random Forest is an **ensemble learning algorithm** that builds many decision trees and combines their predictions to obtain a more stable and accurate model. A collection of weak learners can form a strong learner when their predictions are aggregated.

Each tree is trained on a **different random subset of the data** and uses a **random subset of features** when making splits. This randomness ensures that the trees are diverse.

The final prediction is obtained by aggregating the predictions from all trees:

- **Classification**: Majority voting
- **Regression**: Average of predictions

Random Forest primarily aims to **reduce variance** while maintaining the **low bias** of decision trees.

Decision Trees generally have:

- **Low bias**
- **High variance**

Small changes in the training data can produce very different trees, making them unstable and prone to overfitting.

Random Forest reduces this variance by:

- Training many trees on different subsets of data
- Aggregating their predictions

Averaging across multiple trees stabilizes the model and improves generalization.

Two main sources of randomness are introduced to make trees less correlated.

- **Bootstrap Sampling (Row Randomness)**
  Random samples of the dataset are created **with replacement**. Each tree is trained on one such bootstrap sample.
- **Random Feature Selection (Column Randomness)**
  At every split in a tree, only a **random subset of features** is considered instead of all features.
  Typical choices:
  - Classification: $m = \sqrt{p}$
  - Regression: $m = \frac{p}{3}$

  where p is the total number of features.

This randomness helps reduce correlation between trees, which improves the effectiveness of averaging.

11.1 Out-of-Bag Score (OOB Score)

Since bootstrap sampling is done with replacement, some data points are not included in the training set of a given tree.

- About **63%** of the data appears in a bootstrap sample.
- The remaining **37%** is not used to train that tree.
- These unused samples are called **Out-of-Bag (OOB)** samples.

Each data point can be predicted using the trees that did not see it during training.
The aggregated predictions from these trees are compared with the true labels to compute the **OOB Accuracy**.
This provides an internal estimate of model performance without needing a separate validation set.

11.2 Feature Importance

Random Forest can estimate the importance of features based on how much they reduce impurity across all trees.
Common approaches:

- **Mean Decrease in Impurity (MDI)**
- **Permutation Importance**

Features that frequently produce large impurity reductions are considered more important.

11.3 Proximity Matrix

The **Proximity Matrix** measures how often two data points fall into the same leaf node across all trees in the forest.

$$P(i,j) = \frac{\text{Number of trees where } i \text{ and } j \text{ land in the same leaf}}{\text{Total number of trees}}$$

It acts as a **data-driven similarity measure**.
Applications:

- Clustering
- Missing value imputation
- Outlier detection

If a data point has missing values, we can find the most similar points (high proximity) and use their known values to estimate the missing ones.
Outliers tend to rarely appear in the same leaf nodes as other data points.
An outlier score can be defined as:

$$\text{Outlier Score}(i) = \frac{1}{\sum_j P(i,j)^2}$$

- Large score $\rightarrow$ isolated point
- Likely to be an outlier

11.4 Hyperparameters of Random Forest

Important hyperparameters include:

- Number of trees
- Maximum tree depth
- Number of features considered at each split
- Minimum samples required at a leaf node
- Bootstrap sampling option

Increasing the number of trees generally improves performance but increases computational cost.

12 Adaboost Algorithm

- Ensemble algorithm which combines many weak learners into a strong learner by reweighting training samples to focus on hard examples.
- Learners are trained sequentially, and each one pays more attention to examples the previous learners got wrong.
- Weak Learner - A model that performs slightly better than random guessing.
- Finally we combine all models through a weighted vote.

$$\{(x_1, y_1), ..., (x_n, y_n)\}$$
$$y_i \in \{-1, +1\}$$
$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$
$$\text{Final Prediction} = \text{sign}(F(x))$$

- $h_t(x)$ is the weak learner
- α_t is the learner weight
- T is the number of rounds

12.1 Algorithm

- We initialise sample weights $\frac{1}{n}$
- Train h_t to minimize weighted error

$$\epsilon_t = \sum_{i=1}^{n} w_i^{(t)} 1(h_t(x_i) \neq y_i)$$

- Compute the learner weight

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- lower the error higher the influence. If $\epsilon > 0.5$, model is worse than random (discard then)
- We update the sample weights

$$w_i^{(t+1)} = w_i^{(t)} . e^{-\alpha_t y_i h_t(x_i)}$$

- if the prediction is correct then weight decreases
- if the prediction is incorrect then weight increases
- We normalize the weights after this
- Repeat this process T times.
- and we have the final classifier as follows;

$$H(x) = \text{sign} \left(\sum_{t=1}^{T} \alpha_t h_t(x) \right)$$

- In adaboost the most common weak learners are decision stumps. This has very low variance.

Adaboost minimizes the exponential loss;

$$\mathcal{L} = \sum_{i=1}^{n} e^{-y_i F(x_i)}$$

13 Gradient Boosting

Gradient boosting is functional gradient descent in model space.
We optimize a function F(x) by building it additively;

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

Each h_t is a weak learner (usually a small tree)
We want to minimize;

$$\mathcal{L}(F) = \sum_{i=1}^{n} L(y_i, F(x_i))$$

where L is any differentiable loss and this generalizes adaboost immediately.
At iteration t ; Compute the gradient of the loss with respect to the model output;

$$g_i = \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{t-1}}$$

we move in the direction of negative gradient $-g_i$.
Negative gradient is actually residual (MSE)
So we fit ;

$$h_t = \text{arg min}_h \sum_i (-g_i - h(x_i))^2$$

Each weak learner approximates the negative gradient of the loss.
After fitting h_t ;

$$F_t(x) = F_{t-1}(x) + \eta . h_t(x)$$

- why gradient boosting considered functional gradient descent?
- Why does fitting residuals work only for squared error? what replaces residuals for other losses?
- Why does gradient boosting reduce bias but can increase variance?
- Why does shrinkage (learning rate) improve generalization?
- Margin Theory in the context of boosting?
- Exponential loss and logistic loss : which is more sensitive to outliers?

14 XGBoost

XGBoost is a regularised, second order gradient boosting with trees, engineered for scale.
Conceptually;

- Same additive model as Gradient Boosting
- Same idea : fit corrections
- Uses 2nd order Taylor expansion
- Explicit Regularisation
- Optimizes tree structure + leaf values jointly

We build an additive model;

$$\hat{y}_i = F(x_i) = \sum_{t=1}^{T} f_t(x_i)$$

Each f_t is a tree and each tree maps an input to a leaf value.
XGBoost objective is as follows;

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t)$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Here γ is the penalty per leaf (tree complexity) and λ is the L2 penalty on leaf weights. This regularisation term is a huge difference from vanilla GBM.

This penalizes too many leaves (prevents deep trees) and large leaf values - prevents aggressive corrections. Don't add a tree unless it helps enough to justify its complexity.

At iteration t, - step-wise learning (one tree at a time)

$$F^t(x) = F^{(t-1)}(x) + f_t(x)$$

We want to choose f_t to minimize;

$$\mathcal{L}^{(t)} = \sum_i L\left(y_i, F^{(t-1)}(x_i) + f_t(x_i)\right) + \Omega(f_t)$$

What small function should I add to improve predictions?This is the gradient boosting logic.

Regression Process

- We start with an initial prediction (0.5) regardless of regression or classification
- We compute the residuals and fits a regression tree to the residuals (using xgboost tree)
- How we build a xgboost tree for regression ?

Each tree starts with a leaf of all the residuals;

$$\text{Similarity Score} = \frac{\text{Sum of Residuals,Squared}}{\text{Number of Residuals} + \lambda}$$

We cluster similar residuals into two groups.

How much better the leaves cluster similar residuals than the root node? We do this by calculating the Gain of splitting the residuals into two groups.

$$\text{Gain} = (\text{Left}_{similarity} + \text{Right}_{similarity}) - \text{Root}_{similarity}$$

We pick a parameter called gamma (γ) and we compare this with the gain. If the difference between the Gain and gamma is negative (Gain $- \gamma$) , we will remove the branch. This is how we prune the tree. If we increase the gamma, the pruning will be extreme and we may end up in the initial prediction itself.

Lambda, is a Regularization parameter, which means that it is intended to reduce the prediction's sensitivity to individual observations. If lambda is ¿=0, the similarity scores are smaller.

$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$$

The output value equation is like the similarity score except we do not square the sum of the residuals.

Learning rate of xgboost is 0.3 (by default).

We keep building trees until the Residuals are super small, or we have reached the maximum number.

Classification Process

We start with the initial prediction 0.5

We have a new formula for similarity scores in classification.

$$\frac{\text{Sum of Residuals,Squared}}{\sum[\text{Previous Probability} * (1 - \text{Previous Probability})] + \lambda}$$

$$Cover = \sum[\text{Previous Probability} * (1 - \text{Previous Probability})]$$

In regression, cover is just the number of residuals in the leaf.

We need to convert this probability into log(odds) value.

and add this with the tree output; then we get the log(odds) value.

Now we convert this into probability. and we got new residuals.

15 Catboost lgbm

LightGBM is optimized for speed and large-scale datasets by changing how trees are built and how splits are evaluated, while CatBoost focuses on handling categorical features correctly and reducing training bias.

- LightGBM uses **histogram-based splits**, where continuous features are bucketed into bins, making split finding faster and more memory efficient
- LightGBM grows trees **leaf-wise instead of level-wise**, always splitting the leaf with the highest loss reduction, which leads to faster convergence but requires strong regularization to avoid overfitting
- LightGBM handles **missing values natively** by learning the optimal direction for missing data during training

CatBoost is designed to work well with categorical data and small-to-medium datasets by reducing target leakage and prediction bias.

- CatBoost handles **categorical features natively** without requiring one-hot or manual target encoding
- It uses **ordered target statistics**, computing category encodings using only past data to avoid target leakage
- CatBoost applies **ordered boosting**, which reduces prediction shift by computing gradients in an unbiased way
- It builds **symmetric (oblivious) trees**, using the same split at each depth, improving regularization and inference speed
- LightGBM is preferred for very large datasets and high-dimensional data where training speed is critical
- CatBoost is preferred when datasets contain many categorical features and when minimal preprocessing is desired

16 Clustering Evaluation

16.1 Silhouette Score

For each data point, the score compares the average distance to other points within the same cluster (cohesion) to the average distance to points in the nearest cluster (separation). The Silhouette score is calculated for all points in the dataset, and the overall score is the average of individual point scores.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $a(i)$ is the **average distance** between point i and all other points in the same cluster. This measures cohesion.
- $b(i)$ is the **minimum average distance** between point i and all points in the nearest cluster. This measures separation.
- $\max(a(i), b(i))$ is used to normalize the score, ensuring that it falls between -1 and 1.
- The **Silhouette score** $s(i)$ can take values between -1 and 1:
- A score close to **1** indicates that the point is well clustered (both close to its own cluster and far from others).
- A score close to **0** indicates that the point lies on or near the boundary between two clusters.
- A score close to **-1** indicates that the point may have been assigned to the wrong cluster.

The **Silhouette score for the entire dataset** is the average of the Silhouette scores of all individual points:

- **Assumes Convex Clusters**: The Silhouette score assumes that clusters are convex and isotropic, which may not always be the case, especially for non-globular clusters.
- **Sensitive to Outliers**: The score can be sensitive to outliers or noise, as these points can distort the cohesion and separation calculations.

17 KMeans Clustering

K-means clustering is an unsupervised learning algorithm that partitions a dataset $X = x_1, x_2, ..., x_n$ into K disjoint clusters $C = c_1, c_2, ..., c_k$, where each clusters is represented by its centroid (mean) μ_k, with the objective of minimizing the within-cluster sum of squares.

In K Means we partition data into K groups such that ;

- Each data point belongs to exactly one cluster (hard clustering)
- Clusters are compact (within cluster sum of squares should be as less as possible)
- Each cluster is represented by a centroid(mean of the cluster points)

Group data into K clusters so that points are close to their own cluster centre and far from others

- Inputs : Dataset with n data points and the number of Clusters : K
- Initialise centroids (randomly or K-Means $++$)
- Assignment Step : For each data point, compute its distance to each centroid and assign the point to the nearest centroid.
- For each cluster - recalculate the centroid as the mean of all points assigned to that cluster.
- Repeat this process, until the centroids do not change significantly or assignments do not change or maximum number of iterations is reached.
- Final Outputs : The data points and the associated cluster labels.

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

KMeans is **Coordinate Descent**- assignment step is there for optimize the cluster labels, and update step is there for optimizing the centroids

Concerns- regarding cluster shape, selection of distance metric, selection of evaluation metric, selection of number of clusters, scaling of data points, centroid initializations,

Computational Complexity

Question- Given a set of points assigned to a cluster, where should the centroid be placed so that the total squared distance from the points to the centroid is minimized? The centroid that minimizes the sum of squared euclidean distances is the mean of the points.

17.1 K-Means has local minima

- The objective is non-convex
- Cluster assignments are discrete (combinatorial)
- The algorithm uses greedy alternating optimization
- Initialization fixes the "basin of attraction"

Once K-Means makes early assignment decisions, it cannot undo them globally - it only makes locally improving moves.

- The number of possible clustering grows exponentially with data size.
- Even if the objective is convex with respect to centroids, it is not convex with respect to assignments.
- This is why K-Means $++$ helps (but doesn't solve it): improves the initialization, reduces the probability of bad local minima.

Pairwise distance to centroid based kmeans objective

- Pairwise View : All points inside a cluster should be close to each other
- All points should be close to the cluster center
- Minimizing the sum of squared distances between all pairs of points inside a cluster is equivalent to minimizing the sum of squared distances from points to the cluster centroid.

17.2 Variance Decomposition

$TSS = WCSS + BCSS$. Since TSS is constant for any given dataset, minimizing $WCSS$ is mathematically equivalent to maximizing $BCSS$. In other words, creating tight clusters automatically means creating well-separated clusters. - complementary

17.3 Computational Complexity

In the assignment step, we must compute the distance from each of the n points to each of the K centroids. For data in d dimensions, each distance computation requires $O(d)$ a operations. Thus, the assignment step has complexity $O(nKd)$

In the update step, we must compute the mean of each cluster. This requires summing all points in each cluster and dividing by the cluster size, which takes $O(nd)$ operations in total.

If the algorithm converges in I iterations, the total time complexity is ; $O(I.n.K.d)$

17.4 K-Means ++

K-means++ Initialization- The key idea is to choose initial centroids that are far apart from each other.

- Pick one data point uniformly at random.
- For each data point , compute distance to nearest already chosen centroid.

$$D(x_i) = \min_{1 \le j \le t} \|x_i - \mu_j\|$$

- When computing this distance, we are asking the question 'Is this point already well represented by an existing centroid'. if yes then the $D(x_i)$ will be small, otherwise large. Points with large distance are good candidates for new centroids.
- Instead of always picking the farthest point (deterministic) K means ++ uses probabilistic sampling.

$$P(x_i) = \frac{D(x_i)^2}{\sum_{j=1}^{n} D(x_j)^2}$$

Why does kmeans ++ reduce sensitivity to outliers?- Because it samples points proportional to squared distance, so dense regions collectively outweigh isolated extreme points, unlike deterministic farthest point selection.

17.5 K-Means Through the Expectation-Maximization Lens

Recalling K-Means objective;

$$\min_{\{z_{ik}, \mu_k\}} \sum_{i=1}^{n} \sum_{k=1}^{K} z_{ik} \|x_i - \mu_k\|^2$$

Here z_{ik} is 1 if x_i belongs to cluster k and 0 otherwise.

Expectation Step :
Given current centroids μ_k;

$$z_{ik} = \begin{cases} 1 & k = \arg\min_j \|x_i - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases}$$

This is hard assignment ; Each point belongs to exactly one cluster(Probabilities are 0 or 1)

Maximization Step Given assignments z_{ik};

$$\mu_k = \frac{\sum_{i=1}^{n} z_{ik} x_i}{\sum_{i=1}^{n} z_{ik}}$$

This is called the mean update

17.6 Comparison of K-Means & GMM in terms of EM Algorithm

KMeans and GMM both aim to cluster data, but they differ in what they assume about the data.

Kmeans focuses directly on partitioning the data into groups by minimizing distances. It does not try to model how the data was generated; it only cares about assigning each point to the closest cluster center. In this sense, it behaves like a discriminative method.

GMM assumes that the data was generated by a mixture of several underlying sources, and each data point is produced by one of these sources. The goal is to learn these sources and estimate how likely each data point came from each one. Because of this, GMM are generative ; instead of just assigning clusters, they try to model the full data distribution and explain the data-generation process.

Kmeans is build around a very simple idea ; Represent a group of points by a single representative point ; what does central even mean . The answers depends on the distance we use. Under Euclidean distance squared, something magical happens; The point that minimize the total squared distance to all points is the mean. This is not true for most other distances.

What breaks if we don't use euclidean distance

- Mean is no longer optimal centroid
- No variance interpretation
- No EM interpretation

What happens if we use Manhattan distance (L1)

Under manhattan distance, the point that minimizes total distance is the median, not the mean. Manhattan distance cares about absolute deviations. The median minimizes absolute error. This is why medians are robust to outliers. So clustering with Manhattan distance naturally leads to k-medians.

17.7 K-Medoid : When the Center Must be a real point

In many real-world cases, distances are not euclidean, data will be mixed or categorical. You want a real data point as the representative. You want robustness to outliers. Medoid is an actual data point whose total distance to other points in the cluster is minimal.

$$\min_{\{m_k\}} \sum_{k=1}^{K} \sum_{x_i \in C_k} d(x_i, m_k)$$

- Choose K data points as initial medoids (random or heuristic)
- Assign each point to the nearest medoid using the chosen distance
- For each cluster;
- Try every point in the cluster as a candidate medoid
- Choose the one that minimizes total distance to others. (new medoid)
- Repeat until convergence
- We don't need any geometry for the distance metric (as long as smaller distance = more similar)
- Expensive : Updating medoids requires pairwise distances. So in practice, people use; PAM (Partitioning Around Medoids) , CLARA (sampling-based), CLARANS (randomized)

17.8 K-Modes (For categorical data)

- For categorical attribute, distance is total Number of mismatched attributes.
- The centroid is the mode;- Which is computed for each attribute.

17.9 K-Prototype(Numeric + Categorical)

- Combines k-means and k-modes in a single objective
- Use Euclidean distance for numeric features
- Use Mismatch distance for categorical features
- Balance then with a weight

$$d(x, \mu) = \sum_{j \in \text{num}} (x_j - \mu_j)^2 \; + \; \gamma \sum_{j \in \text{cat}} \mathbf{1}(x_j \neq \mu_j)$$

- The centroid has two parts: Numeric attributes will be updated using the mean. Categorical part will be updated using the mode.
- Think of γ as answering : How much numeric difference is equivalent to one categorical difference?

17.10 Assumptions of KMeans

- Clusters are compact and well separated
- Variance within clusters is roughly similar
- Features are on comparable scales

- Euclidean geometry is meaningful
- Clusters are convex

Scaling and High-Dimensional Behaviour

- Curse of dimensionality effect on Euclidean distance
- PCA can be applied

18 DBSCAN

DBSCAN is a **density-based clustering algorithm**. It groups together points that lie in **dense regions** of the data and marks points in **sparse regions as noise (outliers)**.

Key advantages compared to K-Means:

- No need to specify the number of clusters beforehand.
- Can detect **arbitrarily shaped clusters**.
- Naturally identifies **outliers**.

18.1 Key Parameters

- ϵ **(Epsilon)** Maximum distance between two points for them to be considered neighbors.
- **MinPts (Minimum Points)** Minimum number of points required in the ϵ-neighborhood for a point to be considered a core point.

18.2 Types of Points

Let $N_\epsilon(p)$ denote the ϵ-neighborhood of point p.

- **Core Point** A point p is a core point if:

$$|N_\epsilon(p)| \geq \text{MinPts}$$

- **Border Point** A point that is within the ϵ-neighborhood of a core point but does not itself have enough neighbors to be a core point.
- **Noise Point** A point that is neither a core point nor reachable from any core point.

18.3 Density Reachability

A point q is said to be **directly density reachable** from point p if:

- q lies within the ϵ-neighborhood of p
- p is a core point

Density reachability allows clusters to grow outward from core points.

18.4 Density Connectivity

Two points p and q are **density connected** if there exists a chain of core points linking them.

Clusters in DBSCAN are formed from groups of density-connected points.

18.5 DBSCAN Algorithm

- Select an unvisited point p.
- Find all points in its ϵ-neighborhood.
- If p is a **core point**:
  - Start a new cluster.
  - Recursively add all points density-reachable from p.
- If p is not a core point:
  - Temporarily label it as noise (it may later become a border point).
- Repeat until all points have been visited.

18.6 Choosing ϵ

A common method is the **k-distance plot**:

- Compute the distance to the k-th nearest neighbor for each point.
- Sort these distances.
- Plot the values.
- The point where the curve shows a sharp bend (elbow) suggests a good ϵ value.

18.7 Advantages

- Finds clusters of arbitrary shapes.
- Does not require specifying the number of clusters.
- Robust to outliers.

18.8 Limitations

- Performance depends heavily on the choice of ϵ and MinPts.
- Struggles with datasets having clusters of very different densities.
- Computationally expensive for very large datasets without spatial indexing.

19 Hierarchical Clustering

Hierarchical clustering builds a **tree-like structure (dendrogram)** of the data points, showing how points or clusters are merged (or split) at different distances.

19.1 Basic Idea

The goal of hierarchical clustering is to reveal the **nested grouping structure** of the data. Instead of producing a single partition of clusters (like K-Means), it produces a **hierarchy of clusters**.

- Two main types:
- **Agglomerative (Bottom-Up)** – start with each point as its own cluster, merge iteratively
- **Divisive (Top-Down)** – start with all points in one cluster, split iteratively

Most commonly used: **Agglomerative Clustering**

19.2 Distance Metrics and Linkage Criteria

Hierarchical clustering requires two components:

- **Distance Metric**
  Measures dissimilarity between individual data points.
  Examples:
  - Euclidean distance
  - Manhattan distance
  - Cosine distance
- **Linkage Criteria**
  Defines how the distance between two clusters is computed.
  - **Single Linkage**
    Distance between the **closest pair of points** from the two clusters.
  - **Complete Linkage**
    Distance between the **farthest pair of points** from the two clusters.
  - **Average Linkage**
    Average distance between **all pairs of points** across the two clusters.
  - **Ward's Method**
    Merges clusters that produce the **smallest increase in total within-cluster variance**.

19.3 Agglomerative Clustering Algorithm

- **Start**: Each data point forms its own cluster.
- **Compute Distances**: Calculate distances between all clusters.
- **Merge**: Find the two closest clusters and merge them.
- **Update Distances**: Recompute distances between the new cluster and all other clusters based on the chosen linkage rule.
- **Repeat**: Continue merging until:
  - all points form a single cluster, or
  - a stopping condition is reached (e.g., desired number of clusters).

19.4 Dendrogram

A **dendrogram** is a tree diagram that visualizes the hierarchical clustering process.

- Leaves represent individual data points.
- Internal nodes represent cluster merges.
- The height of the merge represents the **distance between clusters**.

Clusters can be obtained by **cutting the dendrogram horizontally** at a chosen distance level.

This determines the final number of clusters.

19.5 Ward's Method (Discussion)

Ward's method is different from other linkage methods.

- Instead of only considering distance between clusters, it considers the **increase in within-cluster variance**.
- At each step, it merges the pair of clusters that produces the **smallest increase in total variance**.
- This tends to produce **compact and spherical clusters**.

19.6 Advantages

- Does not require specifying the number of clusters in advance.
- Produces a hierarchical structure of clusters.
- Useful for visualizing cluster relationships using dendrograms.

19.7 Limitations

- Computationally expensive for large datasets.
- Once clusters are merged, the decision cannot be undone.
- Sensitive to noise and outliers.

20 Gaussian Mixture Models

In Gaussian Mixture Models, each cluster is modeled using a **Gaussian (Normal) distribution**.

- Each cluster corresponds to a probability distribution.
- Each data point is assumed to be generated from one of these distributions.
- The identity of the generating distribution is unknown (latent variable).

20.1 Hard vs Soft Clustering

Hard Clustering

- Each data point belongs to exactly one cluster.
- Clusters do not overlap.
- Example: K-Means.

Soft Clustering

- Each data point belongs to clusters with certain probabilities.
- Instead of assigning a single cluster label, we estimate the **strength of association** between a data point and

each cluster.

Gaussian Mixture Models are a common method for performing **soft clustering**.

A **mixture model** assumes that the observed data is generated from a mixture of several probability distributions.

A Gaussian Mixture Model represents the probability distribution of data as a weighted sum of Gaussian components.

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

where:

- K = number of mixture components
- π_k = mixture weight of component k
- μ_k = mean vector of component k
- Σ_k = covariance matrix of component k

Constraints:

$$\sum_{k=1}^{K} \pi_k = 1$$

Thus a GMM is parameterized by:

- mixture weights π_k
- component means μ_k
- component covariance matrices Σ_k

20.2 The Latent Variable Problem

If we knew which Gaussian generated each observation, parameter estimation would be straightforward.

However, the source distribution for each observation is **unknown**. This hidden variable makes the estimation problem harder.

Let:

- $p(x|c)$ = likelihood of observation x under component c
- $p(c|x)$ = probability that observation x came from component c

Using Bayes' theorem:

$$p(c|x) = \frac{p(x|c)p(c)}{p(x)}$$

These probabilities are called **posterior probabilities** or **responsibilities**.

20.3 Expectation-Maximization (EM) Algorithm

The EM algorithm is an iterative method used to estimate the parameters of the Gaussian mixture model.

It alternates between two steps:

Expectation Step (E-Step)

- Compute the probability that each data point belongs to each Gaussian component.
- These probabilities are called **responsibilities**.
- Each point is therefore **softly assigned** to clusters.

$$\gamma_{ik} = p(c_k|x_i)$$

This represents how likely data point x_i belongs to component k.

Maximization Step (M-Step)

- Update the parameters of each Gaussian distribution using the computed responsibilities.
- Means, covariances, and mixture weights are re-estimated.

Weighted parameter updates:

- Update means using weighted averages.
- Update covariance matrices using weighted variance.
- Update mixture weights based on cluster responsibility totals.

20.4 Intuition Behind EM

The EM algorithm alternates between two tasks:

- **E-step:** Estimate which Gaussian generated each data point.
- **M-step:** Update the Gaussian parameters to better fit the data based on those assignments.

This process is repeated until the parameters converge.

20.5 Interpretation

After convergence:

- Each data point has a probability of belonging to each cluster.
- Clusters are represented by Gaussian distributions.
- The model captures overlapping clusters and complex shapes better than K-Means.

21 Model Explainability

21.1 Shapley Values

21.1.1 Attributing payout to each player

- A group of players forms a coalition and earns a total payout. The goal is to fairly attribute this payout to each player.
- Let N be the set of all players, $n = |N|$, and $v(S)$ be the value (payout) obtained by coalition $S \subseteq N$.

The Shapley value for player i is

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(n - |S| - 1)!}{n!} \left(v(S \cup \{i\}) - v(S)\right)$$

- S : a coalition that does not contain player i.
- $v(S \cup \{i\}) - v(S)$: the **marginal contribution** of player i when joining coalition S.
- $|S|$: number of players in coalition S, and n is the total number of players.
- $\frac{|S|!(n-|S|-1)!}{n!}$: weight corresponding to the probability that the set S appears before i in a random permutation of players.
- Intuition: consider all possible orderings of players, compute the marginal contribution of player i when they join the coalition, and average this contribution over all permutations.

21.1.2 Shapley values in Machine Learning

- Players correspond to **features**, and the payout is the **model prediction** for a single instance.
- If we look at this specific row, how much did each feature contribute to the model output.
- $v(S)$ is defined as the expected model output when only the features in S are known:
- Practically, this is estimated by fixing the features in S and averaging the model prediction over sampled values of the missing features.
- The Shapley values therefore distribute the prediction of one data point among its features.

21.1.3 Shapley Axioms

- Efficiency – All the credit must be fully distributed among the players; nothing is left undistributed.
- Symmetry – If two players contribute in exactly the same way, they must receive the same credit.
- Null Player – If a player adds no value to any group they join, they should receive zero credit.
- Additivity – If you combine two games, each player's credit in the combined game should be the sum of their credits from the individual games.

21.1.4 Prediction decomposition:

Base value = expected prediction over the dataset

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i$$

Each feature's Shapley value tells you:

- its **direction** (positive means it pushes the prediction up)
- its **magnitude** (how strong the push is)
- its **interaction**, captured implicitly by averaging over all possible coalitions

21.2 LIME

Local Interpretable Model-Agnostic Explanations

- Explains individual predictions of any black-box model using a local surrogate model.
- Focuses on answering:
  - Can we trust a prediction? (LIME)
  - Can we trust the model globally? (Submodular Pick)
- Learns an interpretable model locally around the instance of interest.
- Interpretable representation: simplified input such as binary vectors (e.g., word presence, super-pixels).

21.2.1 LIME Algorithm

- Start with an instance x to be explained.
- Convert x into an interpretable representation z.
- Generate perturbed samples around x.
- Obtain predictions $f(x')$ from the black-box model.
- Compute similarity between x and perturbed samples using:

$$\pi_x(z) = \exp\left(-\frac{D(x, x')^2}{\sigma^2}\right)$$

- Assign higher weights to samples closer to x.
- σ is the kernel width. Larger σ assigns higher similarity to distant points.
- Fit a weighted interpretable model $g \in G$ (typically linear).
- Select top k features from g as explanation.
- The explanation is obtained by minimizing:

$$\mathcal{L}(f, g, \pi_x) + \Omega(g)$$

$$\mathcal{L} = \sum_i \pi_x(z_i) \left(f(x_i) - g(z_i)\right)^2$$

- $\Omega(g)$ controls complexity of the surrogate model.

21.2.2 Submodular Pick Algorithm

- Used to obtain a global understanding by selecting a representative set of instances (not features).
- Generate LIME explanations for many instances to form an explanation matrix (instances $\times$ features).
- Compute global feature importance via aggregation of feature weights across all explanations.
- Define a coverage function that measures how well a set of instances captures important features.
- Initialize an empty set of selected instances.
- Iteratively select the instance that provides the highest marginal gain in coverage.
- Ensures diversity by avoiding redundant instances (diminishing returns property).
- Output is a small set of representative instances along with their explanations.

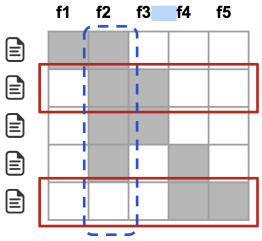

Figure 1: Explanation Matrix : SubModular Pick Algorithm

21.2.3 Advantages / Disadvantages

- Model-agnostic and flexible across data types.
- Provides local, interpretable explanations.
- Useful for debugging and decision transparency.
- Helps explain individual outcomes (e.g., rejection cases).
- Perturbations may generate unrealistic samples.
- Sensitive to kernel width and surrogate model complexity.
- Linear surrogate may fail for highly non-linear decision boundaries.
- Interpretable representation may not capture all behaviors.

21.3 Partial Dependency Plot

- Visual tools to understand how features affect predictions, especially for complex, black-box models
- Shows the average effect of a feature on the model's prediction, marginalizing over all other features.
- Pick a feature X_j
- Choose a grid of values for this feature
- For each such value:
  Replace the feature with that value in all instances.
  Predict using the model
  Average predictions across all instances
  Plot average predictions over the selected grid of values
- Model agnostic
- Example : As age increases, predicted risk increases
- Unrealistic data combinations

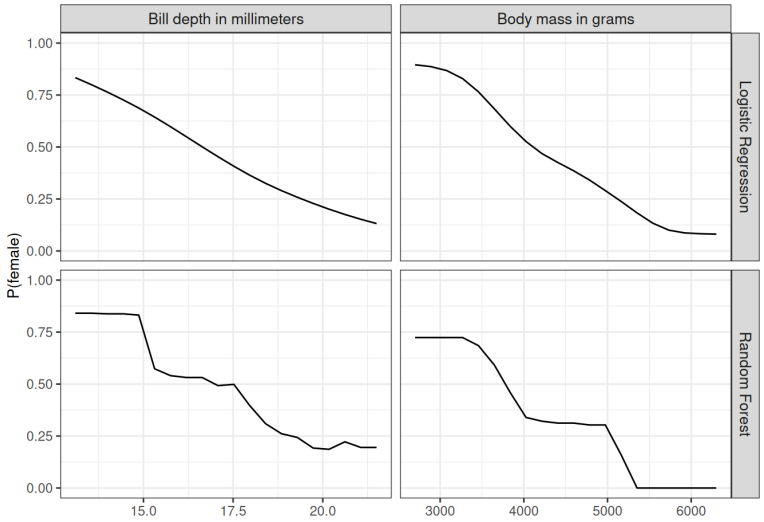

Figure 2: Examples of PDP Plots with respect to two different models

21.4 ICE - Individual Conditional Expectation Plot

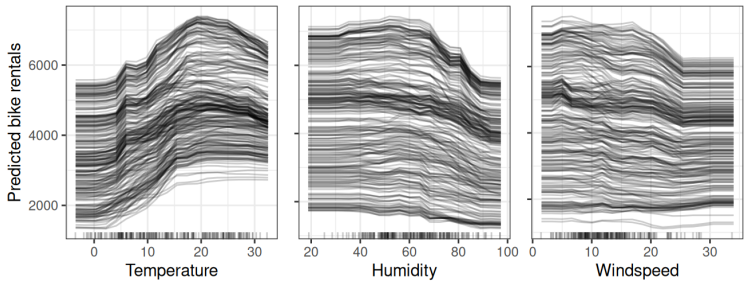

Figure 3: Examples of ICE Plots

- Individual Conditional Expectation (ICE) plots display one line per instance that shows how the instance's prediction changes when a feature changes
- Visualizes the dependence of the prediction on a feature for each instance separately, resulting in one line per instance of a dataset.
- The values for a line (and one instance) can be computed by keeping all other features the same, creating variants of this instance by replacing the feature's value with values from a grid, and making predictions with the black box model for these newly created instances. The result is a set of points for an instance with the feature value from the grid and the respective predictions.

22 Model Calibration

Calibration is the process of ensuring that the predicted probabilities from a model align well with the actual probabilities in the real world

If a classification model outputs a probability of 0.80 for an event occurring, it should mean that on average the event occurs 80

- For 100 customers, the prediction is 0.7 for churn
- So we expect out of this 100, about 70 customers should actually churn.
- If 80 got churn then it was underestimation
- If 50 got churn then it was over estimation

22.1 Steps in Model Calibration

- Train the model on the training set.
- Get predictions (probabilities) on the validation/calibration set.
- Fit the calibration method (Platt scaling, isotonic regression) using predicted probabilities as inputs and actual labels as targets.
- Apply the learned calibration function to the validation set (for checking) the test set (final evaluation) any future unseen data

22.2 Brier Score Loss

Measures mean squared error between predicted probability and the actual label

Overconfident wrong predictions → Higher Brier Score

$$\text{Brier Score} = \text{Reliability - (Resolution} + \text{Uncertainty)}$$

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$$

$$\text{Calibration Error / Reliability} = \sum_{b=1}^{B}\frac{n_b}{N}(p_b - \bar{y}_b)^2$$

$$\text{Resolution/Discrimination} = \sum_{b=1}^{B} \frac{n_b}{N}(\bar{y}_b - \bar{y})^2$$

$$\text{Uncertainty} = \bar{y}(1 - \bar{y})$$

- Resolution checks if the observed proportion differs from the average proportion.
- Reliability checks if average predicted probability matches the actual proportion in that bin.
- Above diagonal → Under Confident
- Below diagonal → Over Confident

22.3 Platt Scaling

Making calibrated probabilities using Platt Scaling

Given the uncalibrated probabilities from my model and the true target labels, I can train a logistic regression on these probabilities to learn the coefficients A and B, and then use the learned sigmoid function

$$P_{\text{calibrated}} = \frac{1}{1 + \exp(Af + B)}$$

to transform the uncalibrated probabilities into calibrated probabilities.

22.4 Isotonic Regression

Isotonic regression is yet another method for getting calibrated probabilities.

Isotonic regression is used when you believe:

- as x increases, y should not decrease.
- Higher dosage → higher (or no-worse) effect
- Higher hours studied → higher score
- Larger sample size → equal or better accuracy
- A risk score → non-decreasing probability
- In real data, noise may break monotonicity (y sometimes goes down when x goes up).
- Isotonic regression fixes that noise.

22.4.1 Why We Need Isotonic Regression

- Used when we know or assume the true relationship between variables should be **monotonic** (non-decreasing).
- Real data may violate monotonicity due to noise.
- Isotonic regression produces a **monotone, smoothed fit** that stays close to the original data.

Typical use cases:

- Machine learning probability calibration.
- Dose–response relationships in biology/medicine.
- Economics (price vs. demand).
- Any setting where monotonicity is known to hold.

22.4.2 What Isotonic Regression Does

- Given sorted data points

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n), \quad x_1 \leq x_2 \leq \cdots \leq x_n$$

- Find fitted values (Uncalibrated probabilities from the model)

$$\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n$$

- That satisfy the **monotonicity constraint**

$$\hat{y}_1 \leq \hat{y}_2 \leq \cdots \leq \hat{y}_n$$

- And minimize squared error

$$\min_{\hat{y}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

22.4.3 Block Averaging Idea

- If a block B of consecutive points must share the same fitted value, the optimal value is

$$\hat{y}_B = \frac{1}{|B|} \sum_{i \in B} y_i$$

- In weighted form (optional):

$$\hat{y}_B = \frac{\sum_{i \in B} w_i y_i}{\sum_{i \in B} w_i}$$

22.4.4 Pool-Adjacent-Violators (PAV) Algorithm

- **Step 1 — Initialization**
- Treat each point as its own block.
- Block $B_i = \{i\}$, value $v_i = y_i$, weight $w_i = 1$
- Here, we have uncalibrated probabilities and actual labels, values are the actual labels.
- **Step 2 — Scan left to right**
- For adjacent blocks B_i, B_{i+1}
- If $v_i \leq v_{i+1}$, continue.
- If $v_i > v_{i+1}$, a **violation** occurs $\rightarrow$ merge blocks.
- **Step 3 — Merge Violating Blocks**
- New block weight:

$$w = w_i + w_{i+1}$$

- New block value:

$$v = \frac{w_i v_i + w_{i+1} v_{i+1}}{w_i + w_{i+1}}$$

- **Step 4 — Backward Check**
- After merging, check previous block.
- If previous block value ¿ new block value $\rightarrow$ merge again.
- Repeat until monotonicity is restored.
- **Step 5 — Form Final Fitted Values**
- Each block produces constant fitted values:

$$\hat{y}_i = \hat{y}_{i+1} = \cdots = \hat{y}_j = v_B \quad \text{for block } B = \{i, \ldots, j\}$$

22.4.5 Interpretation

- Output is a **piecewise-constant, non-decreasing function**.
- It removes random decreases in data while staying close to observations.
- The final fit is the **closest possible monotone sequence** to the original data in squared-error sense.

22.4.6 Numeric Example : Isotonic Regression

Point	Predicted	Label	Calibrated (PAV)
A	0.1	0	0
B	0.2	1	0.6667
C	0.3	1	0.6667
D	0.4	0	0.6667
E	0.5	1	1

- Sort the data by predicted probability. We order the points based on their uncalibrated scores so that the probability axis is monotone, even if the labels are not.
- Start with each point as a block. Each block has a temporary "fitted value" equal to its label.
- Scan left $\rightarrow$ right and find monotonicity violations. A violation occurs when a block on the left has a higher value than the next block on the right.
- Merge violating blocks. When two blocks violate monotonicity, we replace them with a single block whose value is the weighted average of the labels in that block. This average becomes the new calibrated probability for all points in that merged block.

- Backward check. After merging, we check whether the new block violates monotonicity with the block before it. If so, we merge again. This continues until all block values are non-decreasing.
- Final output- Once all violations are resolved, every block has a constant fitted value. These fitted values are the calibrated probabilities,forming a piecewise-constant, non-decreasing function that is optimally close to the true labels under squared error.

22.5 Platt or Isotonic Regression ?

Platt scaling and **isotonic regression** are techniques to calibrate machine learning model probabilities. Platt scaling fits a sigmoid function (effective for SVMs/small data), while isotonic regression fits a non-parametric, monotonic function (better for large data/complex distortions, but prone to overfitting). Key Differences and Selection Criteria

- Platt Scaling (Logistic Calibration):
- Best for: Small calibration datasets (¡ 1000 samples).
- Method: Fits a logistic regression model (sigmoid curve) to the predicted scores.
- Pros: Less prone to overfitting, works well when the distortion is sigmoid-shaped (e.g., SVMs, boosted trees).
- Cons: Limited to monotonic, sigmoid-shaped corrections.
- Isotonic Regression:
- Best for: Large calibration datasets (¿ 1000 samples).
- Method: Fits a non-decreasing, free-form, piecewise-constant function (non-parametric).
- Pros: More powerful, can correct any monotonic distortion.
- Cons: Prone to overfitting on small data.

When to Use Which?

- Use Platt Scaling if your dataset is small or you have limited, high-confidence prior knowledge that the error is sigmoidal.
- Use Isotonic Regression if you have a large dataset, which helps prevent it from fitting noise (overfitting).

23 Feature Selection

23.1 Filter Methods

23.1.1 Mutual Information

- Mutual information is a filter method
- It selected features only on their relationship with the target, not on any trained model.
- A feature is good if it tells you a lot about the target.
- How much knowing X reduces uncertainty about Y.
- Feature and target are independent → MI=0
- Strong dependence → MI is large.

$$I(X,Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

- if features are independent then MI will be 0.

$$H(Y) = -\sum_{y} p(y) \log p(y)$$

$$H(Y|X) = -\sum_{x,y} p(x,y) \log p(y|x)$$

$$I(X;Y) = H(Y) - H(Y|X)$$

- So, MI = Reduction in uncertainty about Y when X is known.

23.1.2 ANOVA For Feature Selection

- Used mainly for classification tasks with a categorical target.
- Does the mean of the feature vary between classes.

- Anova computes the ratio of variance between classes to variance within classes.

$$F = \frac{\text{Variance between classes}}{\text{Variance within classes}}$$

- We have a feature X_j
- Classes $C_1, C_2, \ldots, C_k$
- Number of samples n_i in class i
- $\bar{x}_i$ be the mean of feature in class i
- $\bar{x}$ be the overall mean of the feature
- Compute the between and within class variance and then the f statistic using the following equations.

$$SSB_j = \sum_{c=1}^{k} \sum_{i=1}^{n_c} (\bar{x}_{cj} - \bar{x}_j)^2 = \sum_{c=1}^{k} n_c \left(\bar{x}_{cj} - \bar{x}_j\right)^2$$

$$SSW_j = \sum_{i=1}^{k} \sum_{x \in C_i} (x - \bar{x}_i)^2$$

Between class sum of squares measures how much the class means deviate from overall mean.
Within class sum of squares measure the variability inside each class.

$$F_j = \frac{SSB_j/(k-1)}{SSW_j/(N-k)} \sim F_{k-1,\, N-k}$$

- High $F_j \rightarrow$ Feature strongly separates classes
- Low $F_j \rightarrow$ Feature weakly separates classes
- It only captures differences in mean (ignores variance structure, interactions)

$$\text{Decomposition; } \sum_{j=1}^{k} \sum_{i=1}^{n_j} (x_{ij} - \bar{x})^2 = \sum_{j=1}^{k} n_j (\bar{x}_j - \bar{x})^2 + \sum_{j=1}^{k} \sum_{i=1}^{n_j} (x_{ij} - \bar{x}_j)^2$$

23.1.3 Variation Inflation Factor - VIF

- High VIF $\rightarrow$ Feature is highly correlated with others $\rightarrow$ May consider removing it.

$$VIF_j = \frac{1}{1 - R_j^2}$$

- If X_j feature is highly predictable from other features implies High VIF.
- $1 < VIF < 5$ implies Moderate correlation
- $VIF > 5$ implies Higher correlation, consider removing

23.1.4 Weight of Evidence

How strongly a feature value (or bin) separates good vs bad customers.

$$\text{Weight of Evidence} = \log\left(\frac{\text{\% of Good in the bin}}{\text{\% of Bad in the bin}}\right)$$

- Is this bin more associated with good or bad outcome?
- For continuous features we can use equal width bins or optimal binning.
- After this, we can compute the Information Value ;

$$IV = \sum_i (\text{\% of Good} - \text{\% of Bad}) * WoE$$

- if $IV > 0.3$ then the predictor is strong
- if $IV < 0.1$ its a weak predictor

23.1.5 Chi-Square Test of Independence

- A feature is useful , when its distribution is different across classes.
- A feature is informative when it is dependent of target.
- Null Hypothesis : X & Y are independent
- We create the contingency table.
- We compute the expected counts
- Compute the chi-square statistic

$$E_{ij} = \frac{\text{(Row i Total)} * \text{(Column j Total)}}{N}$$

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

If we have r categories in the feature and c classes in the target column, then the degrees of freedom will be $(r-1)(c-1)$

	Purchase = Yes	Purchase = No	Row Total
Coupon = Yes	40	10	50
Coupon = No	20	30	50
Column Total	60	40	100

	Purchase = Yes	Purchase = No
Coupon = Yes	30 $(= 50 \times 60/100)$	20 $(= 50 \times 40/100)$
Coupon = No	30 $(= 50 \times 60/100)$	20 $(= 50 \times 40/100)$

23.2 Wrapper Methods in Feature Selection

- Iterative procedures
- One independent variable at a time is added or deleted based on the p-value.

23.2.1 Forward Selection

Forward Selection is a greedy feature selection method where features are added sequentially based on performance improvement.

- Start by selecting the single best feature (the one that gives the highest model performance).
- At each subsequent step:
  - Consider adding each remaining feature to the current selected set.
  - Train the model with each possible addition.
  - Evaluate performance using validation or cross-validation.
  - Select the feature that results in the highest performance improvement.
- Add the chosen feature to the feature set.
- Repeat this process iteratively.
- Stop when no further significant improvement is observed or a predefined stopping criterion is met.

23.2.2 Backward Elimination

Backward Elimination is a greedy feature selection method where features are removed sequentially based on their impact on model performance.

- Start with the full set of features.
- At each step:
  - Consider removing each feature one at a time.
  - Train the model on the remaining feature set after each removal.
  - Evaluate performance using validation or cross-validation.
  - Identify the feature whose removal leads to the best performance (or least decrease in performance).
- Remove the selected feature permanently.
- Repeat this process iteratively.
- Stop when removing any further feature significantly degrades performance or a stopping criterion is met.

23.2.3 Stepwise Selection

Stepwise Selection is a hybrid feature selection method that combines forward selection and backward elimination. At each step, it allows both addition and removal of features based on model performance.

- Start with an empty set of features (or optionally a predefined set).
- Perform forward selection:
  - Add the feature that gives the highest improvement in performance.
- After adding a feature, perform backward elimination:
  - Check if any feature in the current set can be removed without degrading performance.
  - Remove the feature whose removal improves or least affects performance.
- Repeat the process:
  - Alternate between adding and removing features.
- Stop when:
  - No feature can be added to improve performance, and
  - No feature can be removed without worsening performance.

23.2.4 Recursive Feature Elimination

- In backward elimination the removal is based on; Which feature can I remove with the least drop in model performance?
- In RFE the removal is based on which feature is least important according to the model itself ?
- so we train model once, get the feature importance scores, coefficients , then remove the weakest, retrain and repeat.
- Backward Elimination: If I remove you, what happens to performance?
- RFE : How important does the model think you are?

23.2.5 Rfecv Algorithm

Recursive Feature Elimination with Cross Validation selects features iteratively.

- Input: Dataset X, y, estimator (model), number of folds k
- Initialize with all features
- Repeat until stopping criterion:
  - Perform k-fold cross-validation:
    - Train the model on training folds
    - Evaluate performance on validation fold
  - Compute average validation score across folds
  - Rank features based on model importance (e.g., coefficients or feature importance)
  - Remove the least important feature(s)
- Track cross-validation score for each number of features
- Select the number of features that gives the best cross-validation performance
- Output: Optimal subset of features

23.3 Embedded Methods in Feature Selection

23.3.1 Lasso - L1 Regularisation

- Feature selection happens automatically during training.
- Add a penalty that pushes coefficients to exactly zero.
- L1 penalty has a diamond shaped constraint.
- Optimal solution often hits the corners.

$$\lambda \sum_j |\beta_j|$$

- As lambda increases small coefficients shrink to 0 first.
- Non-zero coefficients = selected features.

23.3.2 Tree Methods

Tree-based methods perform feature selection during model training by choosing splits based on feature importance.

- At each split, the feature that maximizes a criterion (e.g., Gini impurity, information gain) is selected.
- Irrelevant features are not used in splits and are implicitly ignored.
- Common models: Decision Trees, Random Forests, Gradient Boosted Trees (XGBoost, LightGBM, CatBoost).
- Feature importance is computed via total impurity reduction or gain.
- No separate feature selection step is required.

23.4 Tree-based Feature Importance

- impurity based feature importance
- A feature is important if it is used often to split nodes and the split reduce uncertainty a lot.
- Importance = How much total impurity this feature removes across the whole forest.
- We have impurity measures like Gini, Entropy, Variance(MSE). We can compute the impurity difference between parent and child node. The impurity decrease is the value which is credited to the feature used in that split.
- For a feature we can compute the the entire impurity reductions over the tree. And we can normalize it (based on other feature's importance)
- If we have T trees then we can again take the sum and normalize it.

$$\text{Importance(f)} = \sum_{s \in S_f} \Delta I_s$$

Here Δ_s is the impurity decrease at split s

Following equation gives the feature importance in one tree;

$$\text{Normalized Importance(f)} = \frac{\text{Importance(f)}}{\sum_j \text{Importance(j)}}$$

Now we have T trees ;

$$\text{Importance(f)} = \frac{1}{T} \sum_{t=1}^{T} \text{Importance(f)}$$

And after this we can normalize again to get the value between 0 and 1.

- This approach is biased towards continuous variables.
- High-cardinality categorical features.
- If A and B are correlated ; Tree may pick only one. Other gets low importance even if its meaningful.

23.5 Permutation Feature Importance

- Model agnostic, performance based
- If feature is important, breaking its relationship with the target should hurt performance.
- We do this by ;
  Randomly shuffling one feature column in the validation set.
  Keeping everything else the same
  Measuring how much the model performance drops
  Importance = Performance drop
- Algorithm
  1. Train model on training set
  2. Evaluate on validation set - get M_{base}
  For each feature j:
  Copy dataset
  Randomly permute column j in the validation set
  Predict with trained model
  Compute new performance M_j
  Importance = Difference from baseline
- Rank features by importance $\rightarrow$ Normalize
- We can use repeated permutations also and take the average for computing M_j

- Issues : If two features are correlated , shuffling one might not hurt much, because model can still use the other feature.

23.6 Drop-column

- Remove feature X_j completely from dataset
- Train a new model f_{-j} on remaining features
- Compute importance via performance drop:
- Measures: **true predictive contribution** of X_j
- Model is different after dropping ⇒ allows re-optimization
- **Key Properties**
  - Captures whether feature is **necessary** for prediction
  - Accounts for **feature redundancy** (via retraining)
  - If correlated features exist ⇒ importance may be low
- **Pros and Cons**
  - Suitable for feature selection
  - Model Agnostic
  - Computationally expensive (retrain for each feature)
  - Not explaining the original trained model

24 Information Theory Concepts

24.1 Entropy

In information theory, entropy is a measure of uncertainty or randomness in a set of data. The more unpredictable or random the data, the higher its entropy.

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2(P(x_i))$$

- Probability and surprice is inveserly related.
- when the probability is 1, the surprice needs to be zero, for this reason we can not say that surprice is 1/p. So surprice is defined as $log(1/p)$.
- Entropy can be defined as expected value of surprice.
- In summary, the entropy is a measure of the average amount of surprise or information content associated with a set of outcomes. If all outcomes are equally likely, the entropy is maximized, indicating maximum uncertainty. If some outcomes are more likely than others, the entropy is reduced, reflecting a lower level of uncertainty.

24.2 Cross Entropy Loss

Cross -entropy loss or log loss measures the performance of a classification model whose output is a probability value between 0 and 1. Cross entropy loss increases as the predicted probability diverges from the actual label. In binary classification, where the number of classes is two, cross entropy can be calculated as follows ;

$$-(y * log(p) + (1 - y) * log(1 - p))$$

In multi-classification scenario, we calculate separate loss for each class label per observation and sum the result.

$$-\sum_{c=1}^{M} y * log(p)$$

24.3 Joint Entropy

Joint entropy is a measure of uncertainty associated with a joint probability distribution of two or more random variables. It is an extension of the concept of entropy, which is used to quantify the uncertainty or information

content of a single random variable. Joint entropy takes into account the uncertainty of multiple variables occurring together.

For two random variables, X and Y, with a joint probability distribution $P(X,Y)$, the joine entropy, $H(X,Y)$ is given by:

$$H(X,Y) = -\sum_{i=1}^{m}\sum_{j=1}^{n} P(x_i,y_j)\log_2(P(x_i,y_j))$$

Joint entropy provides a measure of the average amount of uncertainty or information content associated with the joint occurrences of the two random variables. If X and Y are independent, meaning the occurrence of one does not provide information about the other, then the joint entropy is the sum of the individual entropies: $H(X,Y) = H(X)+H(Y)$

However, if X and Y are dependent, the joint entropy will be less than the sum of the individual entropies, reflecting the reduced uncertainty when the variables are considered together.

24.4 Mutual Information

Mutual information is a measure of the amount of information that knowing the value of one random variable provides about another random variable. It quantifies the reduction in uncertainty about one variable when the value of the other variable is known. Mutual information is widely used in information theory, statistics, and machine learning.

for two discreet random variable X,Y, with probability distribution $P(X,Y)$,the mutual information $I(X;Y)$ is given by:

$$I(X;Y) = \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_i,y_j)\log_2\left(\frac{P(x_i,y_j)}{P(x_i)P(y_j)}\right)$$

Mutual information measures the reduction in uncertainty about one variable (x) given knowledge of the other variable(y). If x and y are independent then $I(x,y) = 0$, indicating that knowing the value of y provides no information about x, and vice versa. On the other hand if X and Y are dependent, $I(x,y)$ is positive, indicating that knowledge of one variable can reduce uncertainty about the other.

It is important to note that mutual information is symmetric. It can be used to quantify the relationship between any two random variables.

24.5 KL Kullback-Leibler Divergence

This is also known as relative entropy. Measure of how one probability distribution diverges from a second, expected probability distribution. i its commonly used in information theory to quantify the difference between two probability distributions.

$$D_{\mathrm{KL}}(P\|Q) = \sum_{i} P(i)\log\left(\frac{P(i)}{Q(i)}\right)$$

- KL divergence is not symmetric
- if p and q are same, then kl divergence is zero
- always non-negative
- unbounded

The interpretation of KL divergence is that it measures the extra average number of bits needed to represent events from P using a code optimized for Q, compared to using a code optimized for P. In other words, it quantifies the inefficiency in using the wrong probability distribution Q to represent the true distribution P.

24.6 Population Stability Index

PSI measures how much a variable's distribution has changed between

- a reference dataset (training or baseline)
- a current dataset (new or live)

Has the population changed compared to what the model was trained on ?

- Suppose we have a feature, the first step is to divide it into bins (equal width or equal frequency)
- For each bin compute the proportions from reference data and the current data.

$$PSI = \sum_{Bin}(p_i - q_i)\log\left(\frac{p_i}{q_i}\right)$$

- Here, p is the reference (training or baseline)
- Sum over all bins, then we get the PSI
- If value is < 0.1 then No significant change
- If value is 0.1 to 0.25 then moderate shift
- If value is > 0.25 then major population drift

24.7 Identities and Inequalities

Ranges and Bounds

- $0 \leq H(X) \leq \log|\mathcal{X}|$ (Uniform Distribution)
- $0 \leq H(Y|X) \leq H(Y)$
- $\max H(X), H(Y) \leq H(X,Y) \leq H(X) + H(Y)$
- $0 \leq I(X;Y) \leq \min H(X), H(Y)$
- $D_{KL}(p|q) \geq 0$
- $H(p,q) \geq H(p)$

Core Identities

$$H(X,Y) = H(X) + H(Y|X)$$

$$H(X,Y) = H(Y) + H(X|Y)$$

$$H(X_1,\ldots,X_n) = \sum_{i=1}^{n} H(X_i \mid X_1,\ldots,X_{i-1})$$

$$I(X;Y) = H(X) - H(X|Y)$$

$$I(X;Y) = H(Y) - H(Y|X)$$

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

$$H(p,q) = H(p) + D_{KL}(p|q)$$

Important Inequalities

- $H(Y|X) \geq 0$
- $H(Y|X) \leq H(Y)$
- $H(X,Y) \leq H(X) + H(Y)$
- $H(X,Y) = H(X) + H(Y)$;; iff $X \perp Y$
- $D_{KL}(p|q) \geq 0$
- $I(X;Y) \geq 0$
- $I(X;Y) \leq H(X), \quad I(X;Y) \leq H(Y)$

25 Principal Component Analysis (PCA)

25.1 Goal and Intuition

Principal Component Analysis (PCA) is a technique for **dimensionality reduction**. It transforms the original features into a new set of orthogonal variables called **principal components**.

These components are constructed so that:

- They are **uncorrelated**.
- They capture the **maximum possible variance** in the data.

The components are ordered by importance:

- The **first principal component** captures the largest variance.
- The **second principal component** captures the next largest variance while being orthogonal to the first.
- This continues for all remaining components.

Thus PCA finds new axes that best describe the spread of the data. Before applying PCA, the dataset must be centered.

$$x_i \leftarrow x_i - \mu$$

where μ is the mean of each feature.

Centering ensures that PCA finds directions of maximum variance relative to the origin.

PCA can be understood as projecting data onto a new direction.

Project a vector x onto a **unit vector** u:

$$\text{proj}_u(x) = (u^T x)u$$

- u is a unit vector defining a direction.
- $u^T x$ represents the scalar component of x along that direction.

PCA finds the directions onto which projections of the data have the largest variance.

25.2 Steps of PCA

- Standardize or center the data.
- Compute the **covariance matrix**
  - Diagonal elements represent feature variances.
  - Off-diagonal elements represent covariances between features.
- Compute the **eigenvalues** and **eigenvectors** of the covariance matrix.
- Sort eigenvectors according to decreasing eigenvalues.
- Select the top k eigenvectors.
- Project the data onto these directions.

25.3 Eigenvectors and Eigenvalues

- Eigenvectors define the directions of the principal components.
- Eigenvalues represent the amount of variance captured in those directions.
- Larger eigenvalues correspond to directions with greater variability in the dataset.
- Eigenvectors corresponding to distinct eigenvalues are linearly independent.

25.4 Explained Variance

Each eigenvalue indicates how much variance is explained by its corresponding principal component.

If the eigenvalues are

$$\lambda_1, \lambda_2, ..., \lambda_p$$

then the proportion of variance explained by component k is

$$\frac{\lambda_k}{\sum_{i=1}^{p} \lambda_i}$$

This helps determine how many components should be retained.

25.5 Covariance Matrix and Positive Semi-Definite Property

The covariance matrix has two important properties:

- It is **symmetric**
- It is **positive semi-definite (PSD)**

A matrix A is positive semi-definite if

$$x^T Ax \geq 0$$

for every vector x.

25.6 Why Covariance Matrices are PSD

For a covariance matrix Σ,

$$x^T \Sigma x = Var(x^T X)$$

Variance is always non-negative, therefore

$$x^T \Sigma x \geq 0$$

This proves that covariance matrices are positive semi-definite.

The PSD property guarantees that:

- All eigenvalues of the covariance matrix are **non-negative**.
- Variance captured by principal components cannot be negative.
- Eigenvectors form valid orthogonal directions for projection.

Thus PCA provides a meaningful decomposition of variance in the data.

25.7 Connection to Singular Value Decomposition (SVD)

PCA can also be computed using Singular Value Decomposition.

$$X = U\Sigma V^T$$

In this formulation:

- Columns of V correspond to the principal directions.
- Singular values relate to the amount of variance explained.

Many machine learning libraries compute PCA using SVD because it is numerically more stable.

26 Recommendation Systems

26.1 Goal of Recommendation Systems

A recommendation system tries to solve the following problem:

Given users, items, and interaction data, predict how much a user will like an item they have not interacted with yet.

Examples of interaction data:

- Ratings (1–5 stars)
- Clicks / views
- Purchases
- Likes / skips
- Watch time

These interactions are used to estimate user preferences and recommend relevant items.

26.2 Main Types of Recommendation Systems

- **Content-Based Filtering**
  Recommend items that are **similar to the items a user liked in the past**.
- **Collaborative Filtering**
  Recommend items based on the behavior of **similar users or similar items**.

- **Matrix Factorization**
  A model-based collaborative filtering method that learns **latent representations of users and items**.

26.3 Content-Based Filtering

Content-based filtering recommends items to a user based on the similarity between item features and the user's past preferences. Let $R \in \mathbb{R}^{m \times n}$ denote the user–item interaction matrix, and let $x_i \in \mathbb{R}^d$ represent the feature vector of item i (e.g., genre, keywords, embeddings).

A user profile vector is constructed from the items the user has interacted with. For user u, a common formulation is:

$$p_u = \frac{\sum_{i \in I_u} r_{u,i}\, x_i}{\sum_{i \in I_u} r_{u,i}}$$

where I_u is the set of items rated (or interacted with) by user u.

To estimate the preference of user u for a new item j, compute the similarity between the user profile and the item feature vector.
Cosine similarity:

$$\hat{r}_{u,j} = \text{sim}(p_u, x_j) = \frac{p_u^\top x_j}{\|p_u\|\,\|x_j\|}$$

Alternatively, a linear scoring function can be used:

$$\hat{r}_{u,j} = p_u^\top x_j$$

Thus, items whose features are most similar to the user's learned preference vector are recommended.

This approach relies entirely on item features and does not require other users' data. It is effective when item metadata is rich and informative.

Limitations include inability to capture collaborative signals, dependence on feature quality, and tendency to over-specialize (recommending items too similar to those already seen).

26.4 Collaborative Filtering

Users who behaved similarly in the past will have similar preferences in the future We start with a **user–item interaction matrix**:

$$R = \begin{bmatrix} r_{1,1} & r_{1,2} & \cdots \\ r_{2,1} & r_{2,2} & \cdots \\ \vdots & & \end{bmatrix}$$

This matrix is typically **sparse** since most users interact with only a small fraction of items.
Collaborative filtering methods include:

- User-Based Collaborative Filtering
- Item-Based Collaborative Filtering

26.4.1 User-Based Collaborative Filtering

User-based collaborative filtering predicts a user's preference based on the preferences of similar users. Let $R \in \mathbb{R}^{m \times n}$ denote the user–item rating matrix, where $r_{u,i}$ represents the rating given by user u to item i.

For two users u and v, similarity is computed over the set of commonly rated items I_{uv}.
Cosine similarity:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{u,i}\, r_{v,i}}{\sqrt{\sum_{i \in I_{uv}} r_{u,i}^2}\,\sqrt{\sum_{i \in I_{uv}} r_{v,i}^2}}$$

Pearson correlation:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2}\,\sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}}$$

where $\bar{r}_u$ is the mean rating of user u.

For a target user u, define the neighborhood:

$$N_k(u) = \text{Top-}k \text{ users with highest similarity to } u$$

Prediction of rating for item i:

Basic weighted average:

$$\hat{r}_{u,i} = \frac{\sum_{v \in N_k(u)} \text{sim}(u,v)\, r_{v,i}}{\sum_{v \in N_k(u)} |\text{sim}(u,v)|}$$

Mean-centered prediction:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N_k(u)} \text{sim}(u,v)\, (r_{v,i} - \bar{r}_v)}{\sum_{v \in N_k(u)} |\text{sim}(u,v)|}$$

The similarity acts as a weight, so more similar users contribute more to the prediction. Mean-centering adjusts for user-specific rating biases.

Limitations include sparsity of the rating matrix, cold-start issues for new users or items, and computational cost of similarity calculations.

26.4.2 Item-Based Collaborative Filtering

Item-based collaborative filtering predicts a user's preference for an item based on their ratings of similar items. Let $R \in \mathbb{R}^{m \times n}$ denote the user–item rating matrix, where $r_{u,i}$ represents the rating given by user u to item i.

For two items i and j, similarity is computed over the set of users who have rated both items, denoted by U_{ij}.

Cosine similarity:

$$\text{sim}(i,j) = \frac{\sum_{u \in U_{ij}} r_{u,i}\, r_{u,j}}{\sqrt{\sum_{u \in U_{ij}} r_{u,i}^2}\, \sqrt{\sum_{u \in U_{ij}} r_{u,j}^2}}$$

Pearson correlation:

$$\text{sim}(i,j) = \frac{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)(r_{u,j} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)^2}\, \sqrt{\sum_{u \in U_{ij}} (r_{u,j} - \bar{r}_j)^2}}$$

where $\bar{r}_i$ is the average rating of item i across users.

For a target item i, define the neighborhood:

$$N_k(i) = \text{Top-}k \text{ items most similar to } i$$

Prediction of rating for user u on item i:

Basic weighted average:

$$\hat{r}_{u,i} = \frac{\sum_{j \in N_k(i)} \text{sim}(i,j)\, r_{u,j}}{\sum_{j \in N_k(i)} |\text{sim}(i,j)|}$$

Mean-centered prediction:

$$\hat{r}_{u,i} = \bar{r}_i + \frac{\sum_{j \in N_k(i)} \text{sim}(i,j)\, (r_{u,j} - \bar{r}_j)}{\sum_{j \in N_k(i)} |\text{sim}(i,j)|}$$

Here, similarity between items acts as a weight. Items more similar to the target item contribute more to the predicted rating. Mean-centering accounts for item-specific rating bias.

This approach is generally more stable than user-based filtering when the number of users is large, since item similarities are more consistent. However, it still suffers from sparsity and cold-start issues for new items.

26.4.3 Matrix Factorization

Matrix factorization is a **model-based collaborative filtering technique**.

Instead of computing similarities directly, it learns hidden (latent) factors representing user preferences and item characteristics.

We approximate the user–item matrix R as:

$$R \approx UV^T$$

Where:

- R = user–item interaction matrix $(users \times items)$
- U = user latent factor matrix $(users \times k)$
- V = item latent factor matrix $(items \times k)$
- k = number of latent factors

Prediction:

$$\hat{R}_{u,i} = U_u \cdot V_i^T$$

Each user and item is represented by a vector in a **latent feature space**. The dot product between these vectors estimates how much the user will like the item.

Example latent factors might represent:

- action vs romance
- serious vs comedy
- old vs modern movies

These factors are learned automatically from the data.

26.5 Hybrid Methods

Hybrid recommendation systems combine collaborative filtering (CF) and content-based filtering (CBF) to leverage both interaction data and item/user features. CF captures behavioral similarity across users and items, while CBF utilizes explicit item attributes. Hybridization addresses key limitations such as cold-start, sparsity, and overspecialization.

Weighted Hybrid. In this approach, predictions from CF and CBF are combined linearly:

$$\hat{r}_{ui} = \alpha \cdot \hat{r}_{ui}^{CF} + (1 - \alpha) \cdot \hat{r}_{ui}^{CBF}$$

where $\alpha \in [0, 1]$ controls the contribution of each component. This method operates at the score level and is simple to implement.

Switching Hybrid. A single model is selected based on predefined conditions. For example, CBF is used for new users or items (cold-start), while CF is applied when sufficient interaction data is available. This approach avoids combining models but relies on decision logic.

Feature Augmentation. This method integrates signals at the feature level by using representations learned from one model as inputs to another. Let u and v_i denote user and item embeddings from CF, and x_i denote item features.

Two common formulations are:

$$\hat{r}_{ui} = f([u, v_i, x_i])$$
$$\hat{r}_{ui} = f([p_u, x_i, v_i])$$

where p_u is a user profile vector constructed from item features (e.g., average of features of liked items). The function f represents a learned model (e.g., neural network, linear model) trained on interaction data. This approach enables the model to learn complex, non-linear relationships between collaborative and content signals.

Cascade Hybrid. This approach decomposes recommendation into two stages:

- *Candidate Generation (Recall):* A fast model retrieves a set of top-K candidate items (typically $K \approx 10^2 - 10^3$) using embedding similarity (e.g., nearest neighbor search in latent space).
- *Ranking:* A more expressive model re-ranks the candidates using richer features:

$$\hat{r}_{ui} = f(u, v_i, x_i, c)$$

where c denotes contextual features (e.g., time, device, session information). The recall stage emphasizes computational efficiency, while the ranking stage focuses on accuracy.

Meta-level Hybrid. In this approach, the learned representation from one model is used as input to another model. For instance, a content-based model may generate a user profile vector:

$$p_u = \frac{1}{|I_u|} \sum_{i \in I_u} x_i$$

which is then used within a collaborative model:

$$\hat{r}_{ui} = p_u \cdot v_i$$

Unlike feature augmentation, this method transfers entire learned representations between models rather than combining raw features.

Embeddings and Training. Hybrid models typically rely on embeddings for users and items, which can be obtained via matrix factorization, neural collaborative filtering, or feature-based encoders. These embeddings are often learned jointly with the model f.

Training is performed on interaction data (u, i, y), where y represents explicit ratings or implicit feedback (e.g., clicks). Common objectives include mean squared error for explicit feedback and classification or pairwise ranking losses (e.g., Bayesian Personalized Ranking) for implicit data. Negative sampling is typically employed in the latter case.

Summary. Hybrid methods can be categorized based on how CF and CBF are combined: at the score level (weighted), decision level (switching), feature level (feature augmentation), pipeline level (cascade), or representation level (meta-level). Modern large-scale systems predominantly employ cascade architectures with feature-augmented ranking models.

26.6 Metrics for Recommendation Systems

Evaluation of recommendation systems is performed using **offline metrics** (computed on historical data), **online metrics** (real user behavior), and **system-level metrics** (efficiency and scalability).

Offline Metrics These metrics measure the quality of recommendations based on historical interactions.

Accuracy / Ranking Metrics

- **Precision@K:** Fraction of top-K recommended items that are relevant.

$$\text{Precision@K} = \frac{|\text{relevant items in top K}|}{K}$$

- **Recall@K:** Fraction of relevant items retrieved in the top-K list.

$$\text{Recall@K} = \frac{|\text{relevant items in top K}|}{\text{total relevant items}}$$

- **Hit Rate@K:** Indicator whether at least one relevant item is in the top-K recommendations.

$$\text{Hit@K} = \begin{cases} 1 & \text{if at least one relevant item in top K} \\ 0 & \text{otherwise} \end{cases}$$

- **NDCG@K (Normalized Discounted Cumulative Gain):** Measures ranking quality by accounting for both relevance and position of items. It is computed as:

$$\text{DCG@K} = \sum_{i=1}^{K} \frac{rel_i}{\log_2(i+1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

where rel_i is the relevance of the item at rank i, and IDCG@K is the DCG of the ideal ranking.

Example: Suppose we recommend 5 items to a user and the user clicks only on the 2nd and 3rd items in the list:

  - Recommended list (rank 1–5): [Item1, Item2, Item3, Item4, Item5]
  - Clicked items: Item2 (rank 2), Item3 (rank 3)
  - Relevance scores: $rel = [0, 1, 1, 0, 0]$

Step 1: Compute DCG@5

$$\text{DCG@5} = \frac{0}{\log_2(1+1)} + \frac{1}{\log_2(2+1)} + \frac{1}{\log_2(3+1)} + \frac{0}{\log_2(4+1)} + \frac{0}{\log_2(5+1)} \approx 0 + 0.631 + 0.5 + 0 + 0 = 1.131$$

Step 2: Compute IDCG@5 (ideal ranking)

- Ideal order of clicked items at top: [Item2, Item3, Item1, Item4, Item5] - Relevance scores: $rel = [1, 1, 0, 0, 0]$

$$\text{IDCG@5} = \frac{1}{\log_2(1+1)} + \frac{1}{\log_2(2+1)} + 0 + 0 + 0 \approx 1 + 0.631 = 1.631$$

Step 3: Compute NDCG@5

$$\text{NDCG@5} = \frac{\text{DCG@5}}{\text{IDCG@5}} = \frac{1.131}{1.631} \approx 0.693$$

Interpretation: $NDCG@5 < 1$ because the clicked items are not perfectly ranked at the top. The metric rewards higher placement of relevant items.

Beyond Accuracy

- **Coverage:** Percentage of items that can appear in recommendations.
- **Diversity:** Measures how different the recommended items are from each other, e.g., using pairwise dissimilarity.
- **Novelty:** Rewards recommendations of less popular or new items.
- **Serendipity:** Measures recommendations that are both relevant and unexpected to the user.

Online Metrics These metrics are measured through A/B testing or live deployment.

- **Click-Through Rate (CTR):** Fraction of recommended items that users click on.

$$\text{CTR} = \frac{\text{clicks}}{\text{impressions}}$$

- **Conversion Rate:** Fraction of users who perform a desired action (e.g., purchase, subscription) after recommendation.
- **Engagement Metrics:** Measures such as session length, watch time, number of items interacted with.
- **Revenue / GMV:** Direct business impact of recommendations on sales or profit.

System-Level Metrics These metrics evaluate computational performance and recommendation freshness.

- **Latency:** Time required to generate recommendations per request.
- **Throughput:** Number of recommendation requests the system can handle per unit time.
- **Freshness:** How quickly new items or updated user preferences are incorporated into recommendations.

Summary: Offline metrics focus on prediction and ranking quality (precision, recall, NDCG, hit rate), supplemented by diversity, novelty, and serendipity. Online metrics (CTR, conversion, engagement, revenue) capture actual user behavior and business impact. System-level metrics ensure the recommendation engine operates efficiently at scale. Together, these metrics guide the design, evaluation, and deployment of robust recommendation systems.

27 Methods/Techniques in Machine Learning

27.1 Handling Missing Values

Missing data occurs when some feature values are not observed. Let M_{ij} indicate missingness for feature j in sample i.

Missingness mechanisms:

- MCAR:
  - missingness independent of data
  - Missingness has nothing to do with any data (observed or unobserved)
  - You could delete those rows and still get unbiased results.
- MAR:
  - Missingness depends only on data you can see (observed variables)
  - Younger people are less likely to report their income.
  - You know age, so missing income depends on age (observed).
- MNAR:
  - Tricky one : Missingness depends on unobserved values, so observed data alone is not enough to handle it (If missingness can be explained using observed variables like age, it is MAR, not MNAR.)
  - Students who scored very low marks skip entering their marks.
  - You cannot rely only on observed data
  - This can lead to biased results if not handled carefully.

Simple methods :

- Mean/median/mode imputation (fast but reduces variance artificially)
- Row deletion (valid only under MCAR)

Model-based methods :

- Regression imputation: predict missing feature using others
- KNN imputation: use nearest neighbors

Multiple imputation generates several plausible datasets and **averages results**:

$$\hat{\theta} = \frac{1}{m} \sum_{k=1}^{m} \hat{\theta}^{(k)}$$

Important practice :

- Add missing indicator features
- Fit imputation only on training data

27.2 Data Leakage

Data leakage occurs when training uses information unavailable at prediction time, leading to overly optimistic performance.

- Target leakage: feature contains information derived from Y
- Train-test contamination: preprocessing done before splitting
- Temporal leakage: future data used for past prediction
- In cross-validation, preprocessing must be applied separately in each fold.

27.3 Cross Validation Algorithm

27.3.1 Holdout Validation

The dataset is split into:

- Training set
- Testing set

The model is trained on the training set and evaluated on the testing set.

27.3.2 Leave-One-Out Cross Validation (LOOCV)

For a dataset with n observations:

- Train on $n - 1$ observations
- Test on 1 observation
- Repeat n times

The final error is the average validation error.

27.3.3 Leave-p-Out Cross Validation

In leave-p-out cross validation:

- Leave p observations for testing
- Train on the remaining $n - p$ observations
- Repeat for all possible combinations

27.3.4 Repeated Cross Validation

k-fold cross validation is repeated multiple times using different random splits.
The final performance is the average across repetitions.

27.3.5 Stratified Cross Validation

Each fold preserves the class distribution of the original dataset.
Used mainly for imbalanced classification problems.

27.3.6 Time Series Cross Validation

Temporal order is preserved:

- Train on past observations
- Test on future observations

This prevents data leakage in time series data.

27.3.7 k -fold

Cross-validation estimates generalization error by repeated data splitting.

- **KFold Cross Validation:**
- **Input:** Dataset D with n samples, number of folds k
- Shuffle the dataset randomly
- Split the dataset into k approximately equal-sized folds:

$$D = \{D_1, D_2, \ldots, D_k\}$$

- **For** $i = 1$ to k:
  - Use D_i as the validation (test) set
  - Use the remaining data as the training set:

$$D_{\text{train}} = D \setminus D_i$$

  - Train the model on D_{train}
  - Evaluate the model on D_i and compute performance metric:

$$M_i$$

- Compute the average performance across all folds:

$$M = \frac{1}{k} \sum_{i=1}^{k} M_i$$

- **Output:** Average performance M

27.4 Class Imbalance

- **Data-Level Methods (Resampling):**
  - Oversampling minority class (e.g., SMOTE, ADASYN)
  - Undersampling majority class
  - Hybrid methods (e.g., SMOTE + cleaning techniques)
- **Algorithm-Level Methods:**
  - Class weighting (assign higher penalty to minority class)
  - Cost-sensitive learning (use misclassification cost matrix)
  - Modified loss functions (e.g., weighted cross-entropy, focal loss to focus on hard samples)
- **Threshold Adjustment:**
  - Tune decision threshold based on Precision-Recall or ROC trade-off
- **Evaluation Metrics:**
  - Use appropriate metrics: Precision, Recall, F1-score, ROC-AUC, PR-AUC
- **Ensemble and Advanced Methods:**
  - Balanced ensemble methods (e.g., Balanced Random Forest, boosting)
  - Treat as anomaly detection for extreme imbalance (e.g., Isolation Forest)
  - Data augmentation or synthetic data generation (e.g., GANs)

27.5 Encoding Strategies

- **Basic Encoding Methods:**
  - One-hot encoding for low-cardinality features
  - Ordinal encoding for naturally ordered categories
- **High-Cardinality Handling:**
  - Frequency / count encoding
  - Hash encoding (fixed-dimensional representation)
- **Statistical Encoding:**
  - Replace categories with aggregated statistics (e.g., mean)
  - Use cross-validation and smoothing to avoid overfitting and leakage

- **Model-Specific Techniques:**
  - Tree-based models can work with label encoding or statistical encoding
  - Neural networks use embedding representations
- **Practical Considerations:**
  - Handle rare categories (group into "Other" or regularize)
  - Handle unseen categories using a default/global value

27.6 Feature Transformations

- **Purpose:**
  - Bring features to a comparable scale to improve model performance and convergence
- **Standardization (Z-score scaling):**
  - Transforms data to zero mean and unit variance

$$x' = \frac{x - \mu}{\sigma}$$

  - Suitable for most models (e.g., linear models, SVM, neural networks)
- **Min-Max Scaling (Normalization):**
  - Scales data to a fixed range (usually $[0, 1]$)

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

  - Sensitive to outliers
- **Robust Scaling:**
  - Uses median and interquartile range (IQR)

$$x' = \frac{x - \text{median}}{\text{IQR}}$$

  - More robust to outliers
- **Log Transformation:**
  - Reduces skewness in positively skewed data

$$x' = \log(x + 1)$$

  - Useful for heavy-tailed distributions
- **Other Transformations:**
  - Power transforms (e.g., Box-Cox, Yeo-Johnson) for normality
  - Unit vector normalization (scale samples to unit norm)
- **Practical Notes:**
  - Fit scaling parameters on training data only and apply to test/production data
  - Tree-based models typically do not require feature scaling

27.7 Model Monitoring

Model monitoring tracks changes in data, predictions, and performance to detect model degradation. It is essential because real-world data is non-stationary, and models must adapt to maintain reliability. distributions may change.

- Monitoring Elements
  - Input data distribution:
  - Predictions:
  - Performance:
- Data drift can be measured by PSI and KS Test.
- prediction drift
- concept drift
- Practical Monitoring Signals:
  - Mean and variance of features
  - missing value rates

  - category distribution
  - prediction distribution
- When drift is detected, better we update training data with recent samples or we can retrain or fine-tune the model.